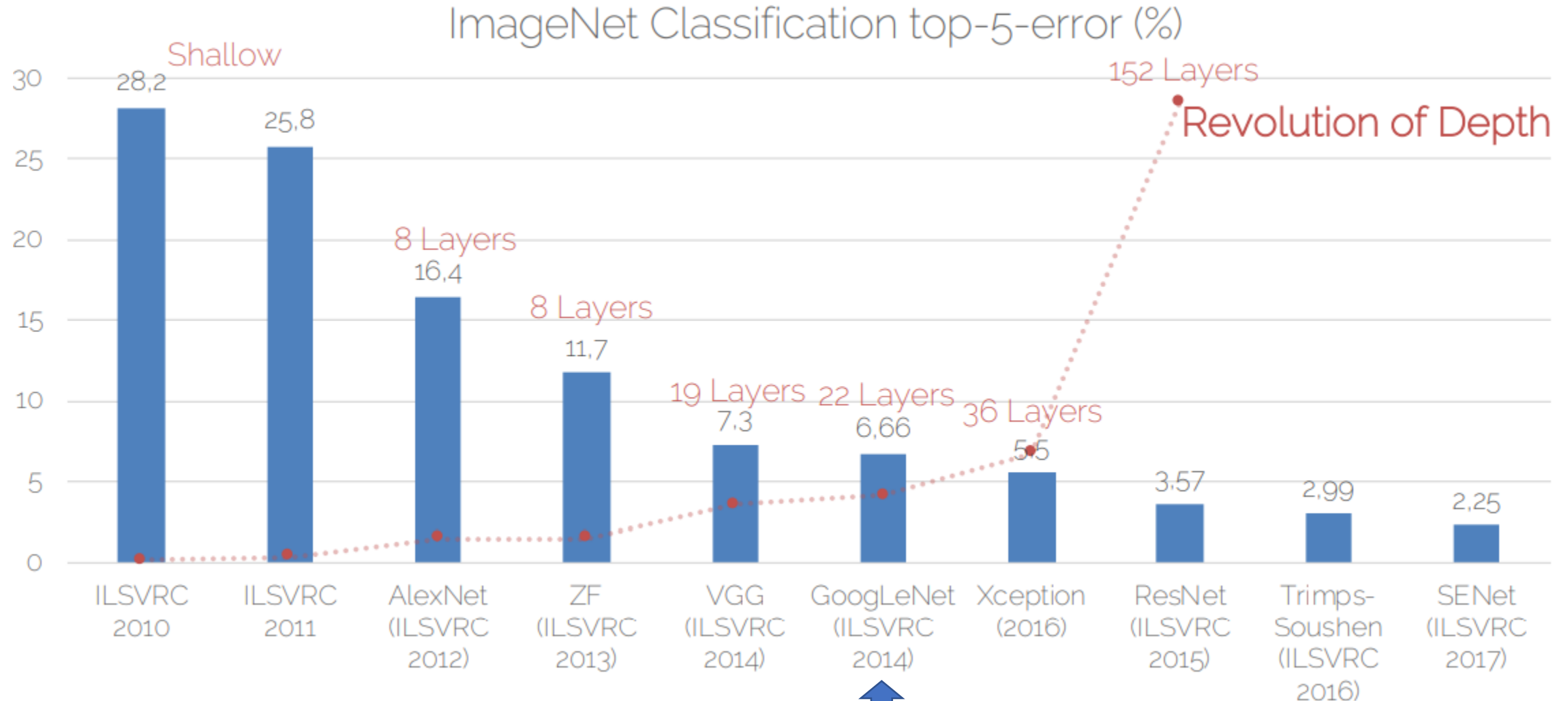


CNN Architectures

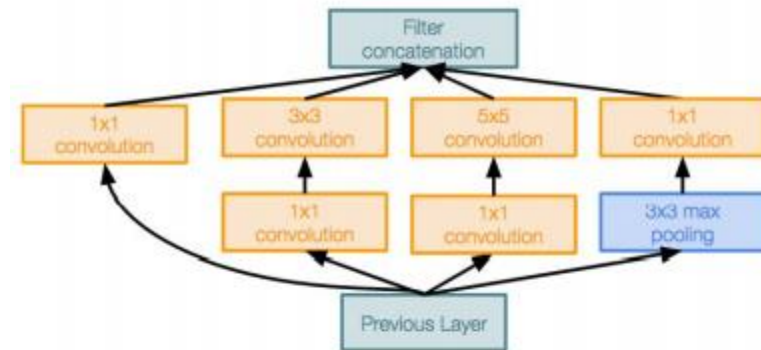
ImageNet Benchmark



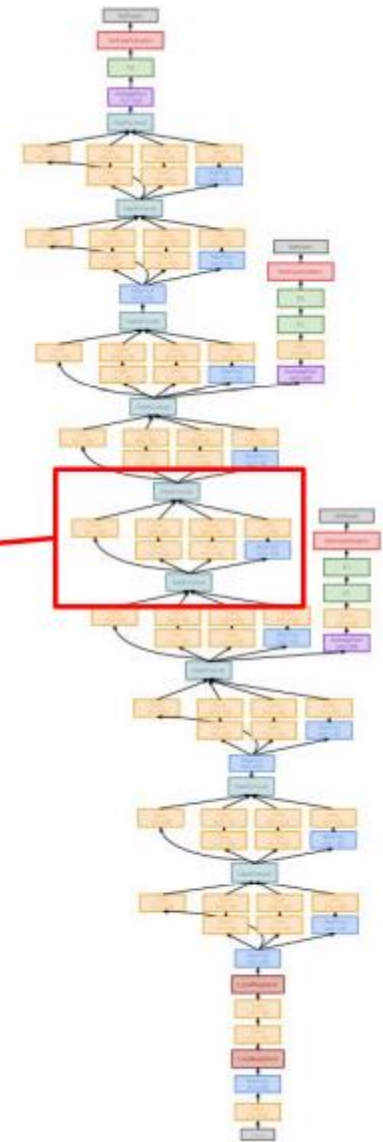
GoogLeNet

Deeper networks, with computational efficiency

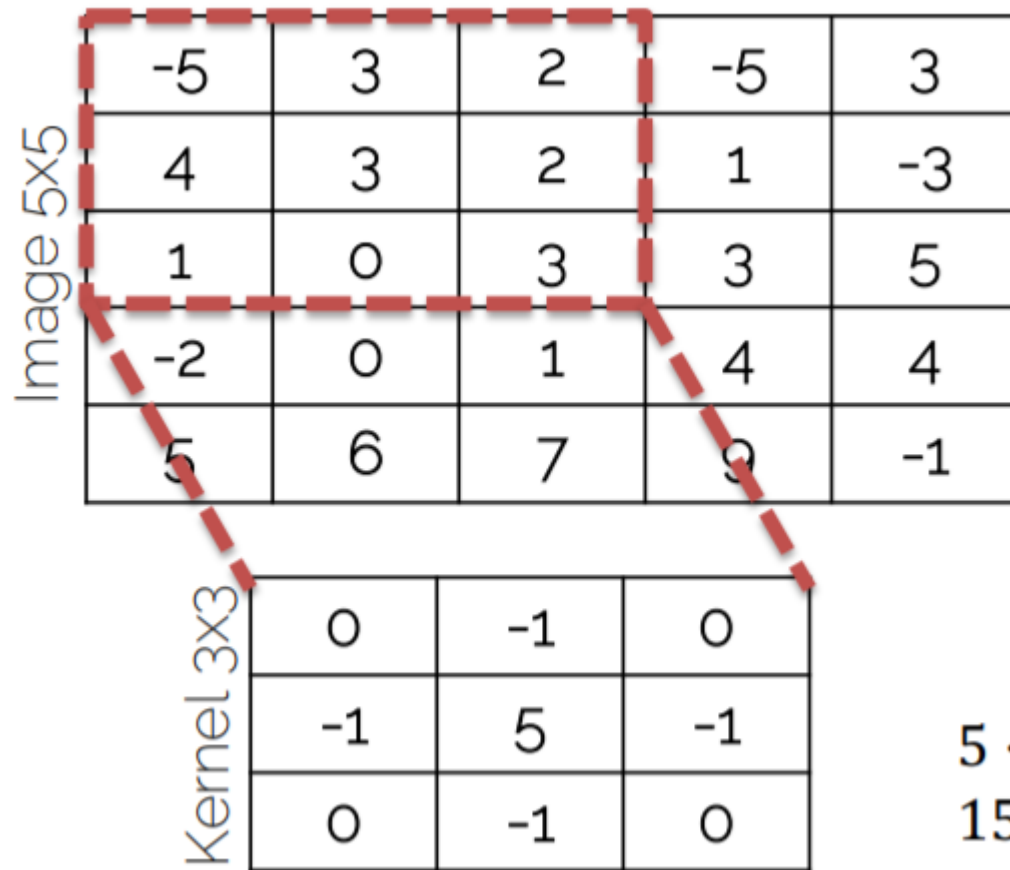
- 22 layers
- Efficient “Inception” module
- Avoids expensive FC layers
- 12x less params than AlexNet
- 27x less params than VGG-16
- ILSVRC’14 classification winner (6.7% top 5 error)



Inception module



Recall: Convolutions on Images

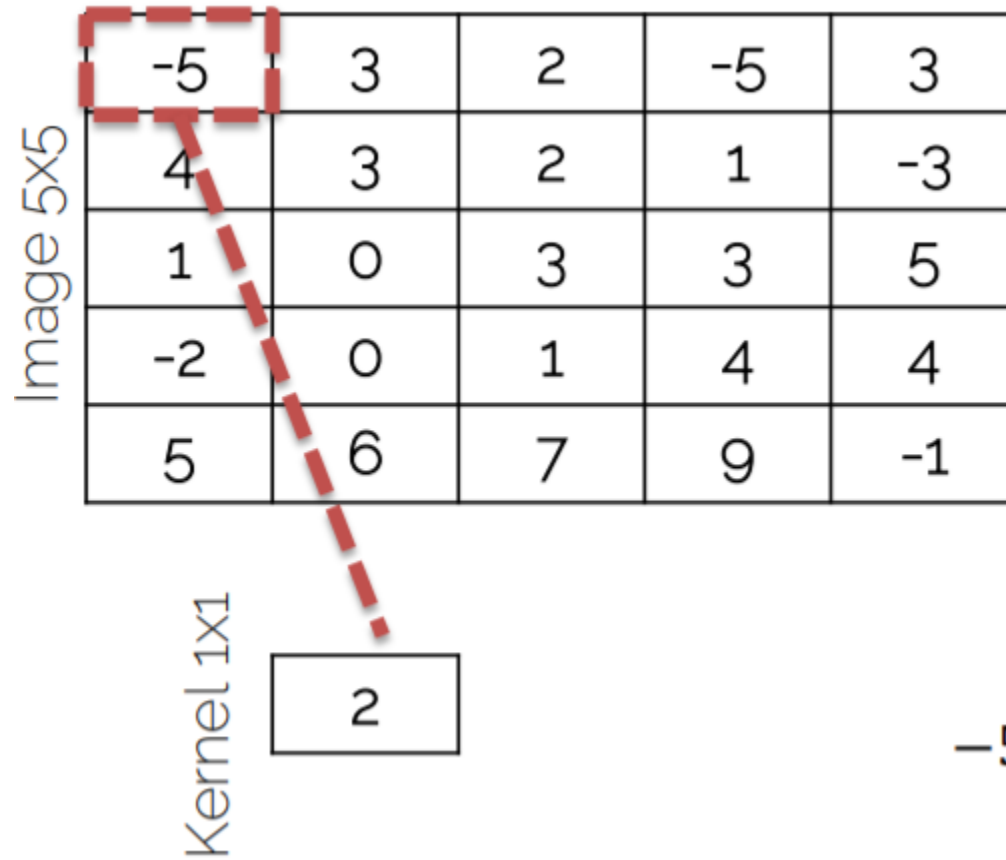


Output 3x3

6		

$$5 \cdot 3 + (-1) \cdot 3 + (-1) \cdot 2 + (-1) \cdot 0 + (-1) \cdot 4 = 15 - 9 = 6$$

1x1 Convolution



-10				

$$-5 * 2 = -10$$

1x1 Convolution

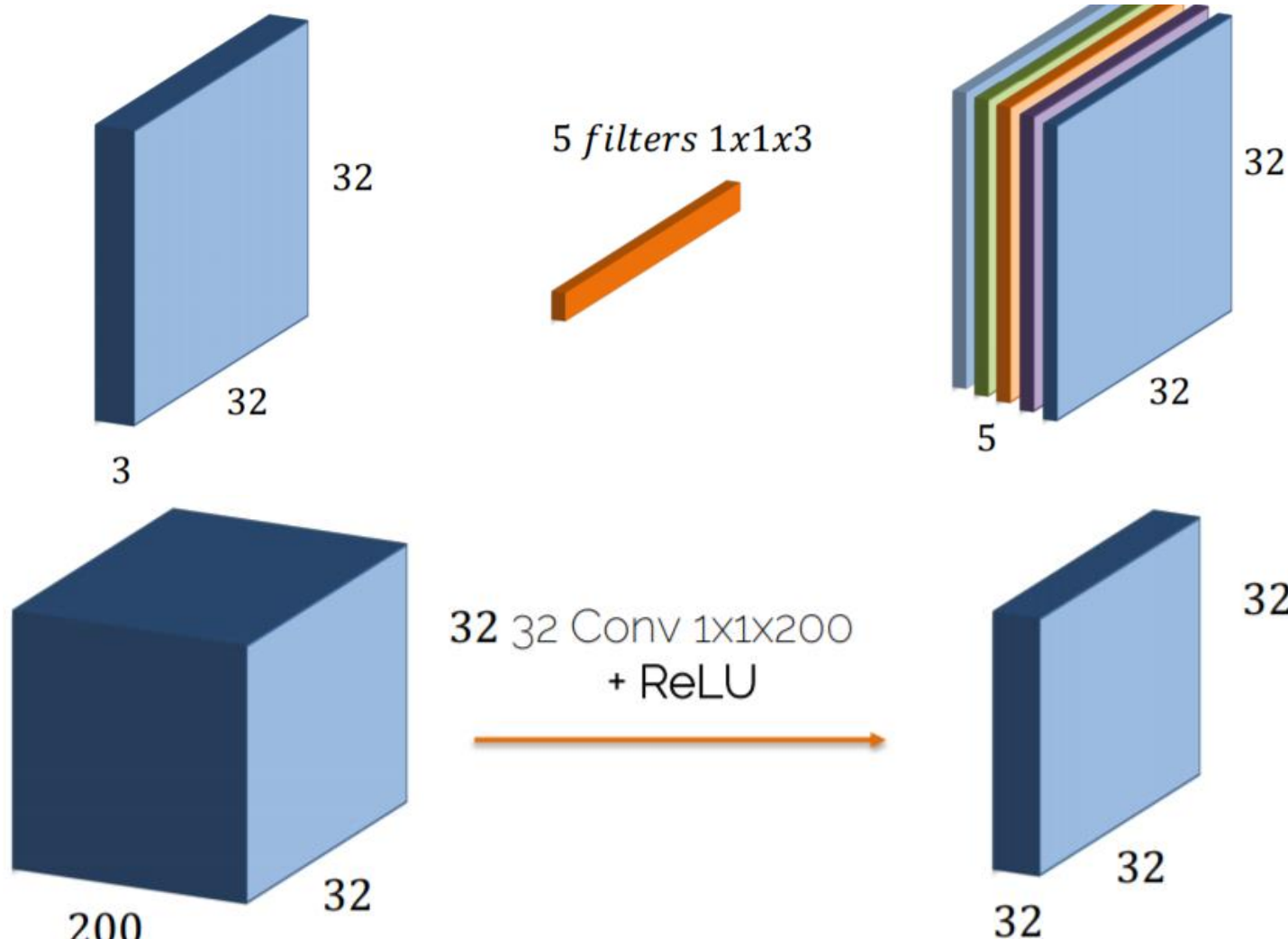
1x1 kernel: keeps the dimensions and scales input

Image 5x5

-5	3	2	-5	3
4	3	2	1	-3
1	0	3	3	5
-2	0	1	4	4
5	6	7	9	-1

-10	6	4	-10	6
8	6	4	2	-6
2	0	6	6	10
-4	0	2	8	8
10	12	14	18	-2

1x1 Convolution

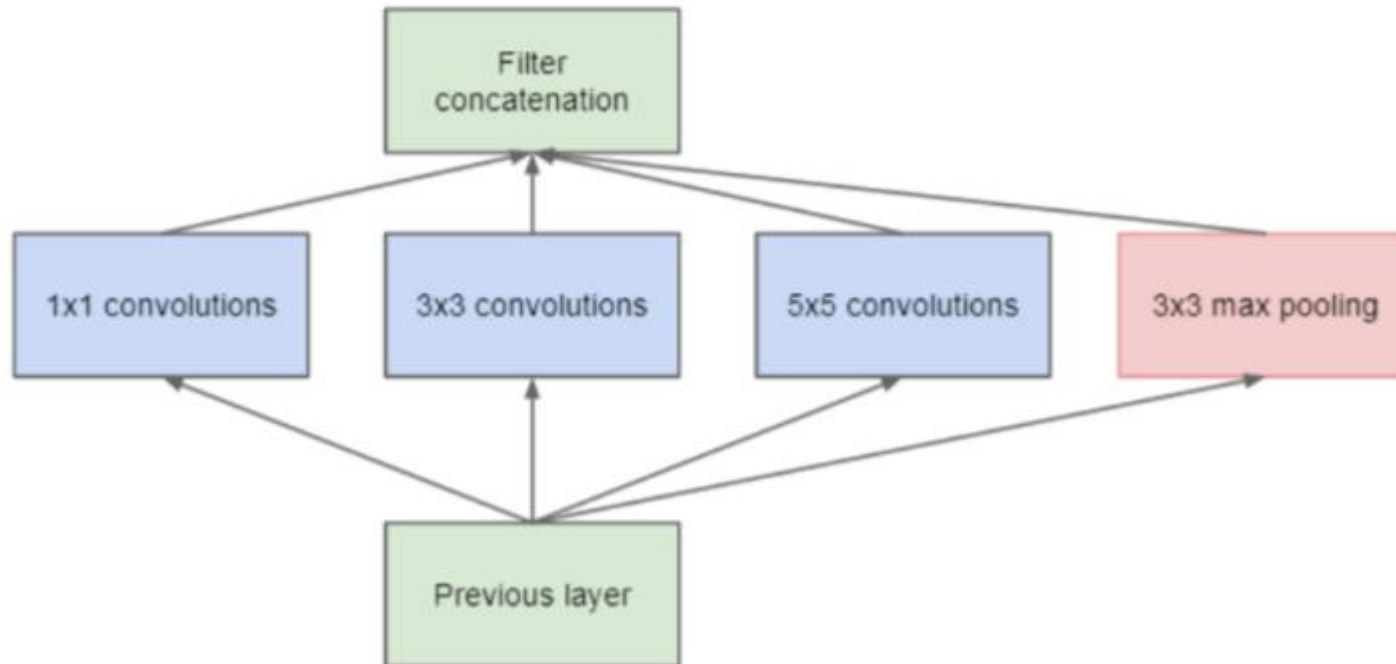


Using more convolutional filters

- Use it to shrink the number of channels
- Further adds a non-linearity. More complex functions can be learned.

Inception Layer

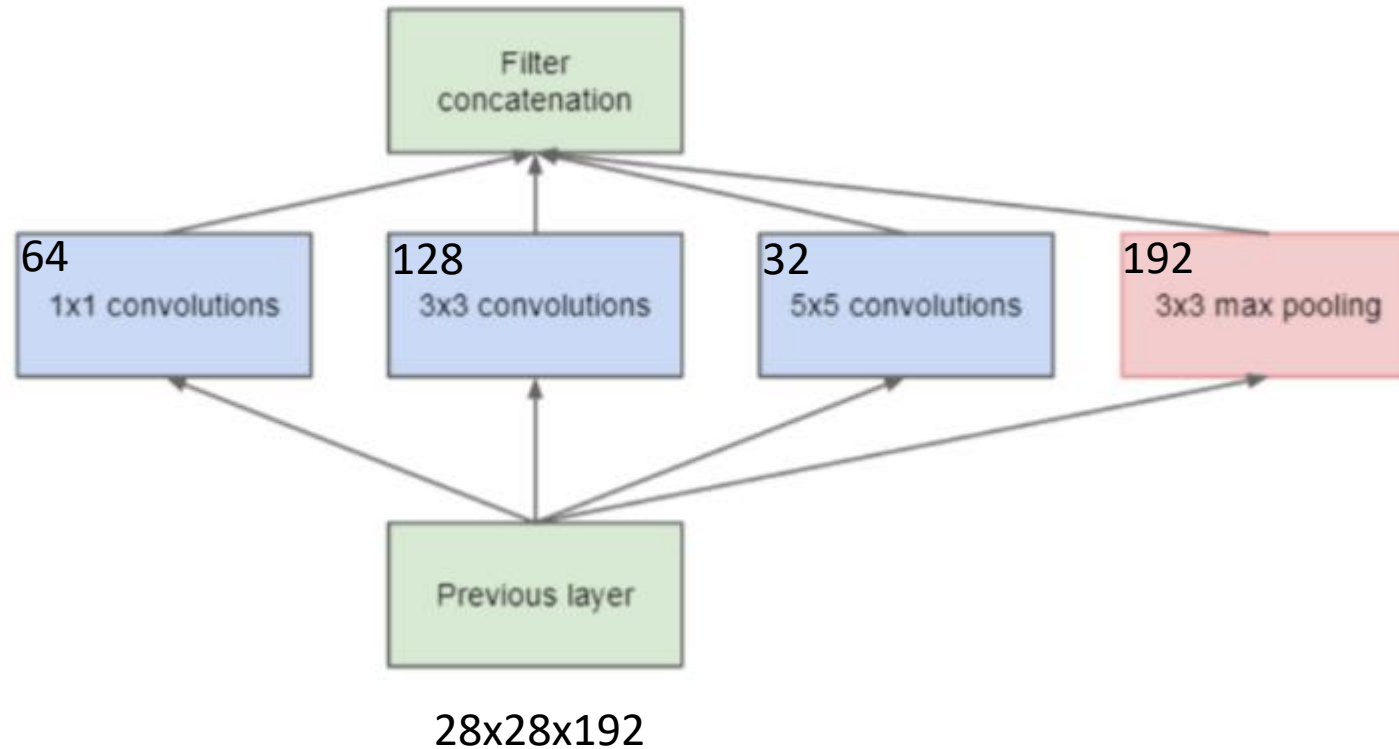
Filters with multiple sizes operating on the same level



Inception Layer

What are the output sizes of all different filter operations?

What is output size after filter concatenation?

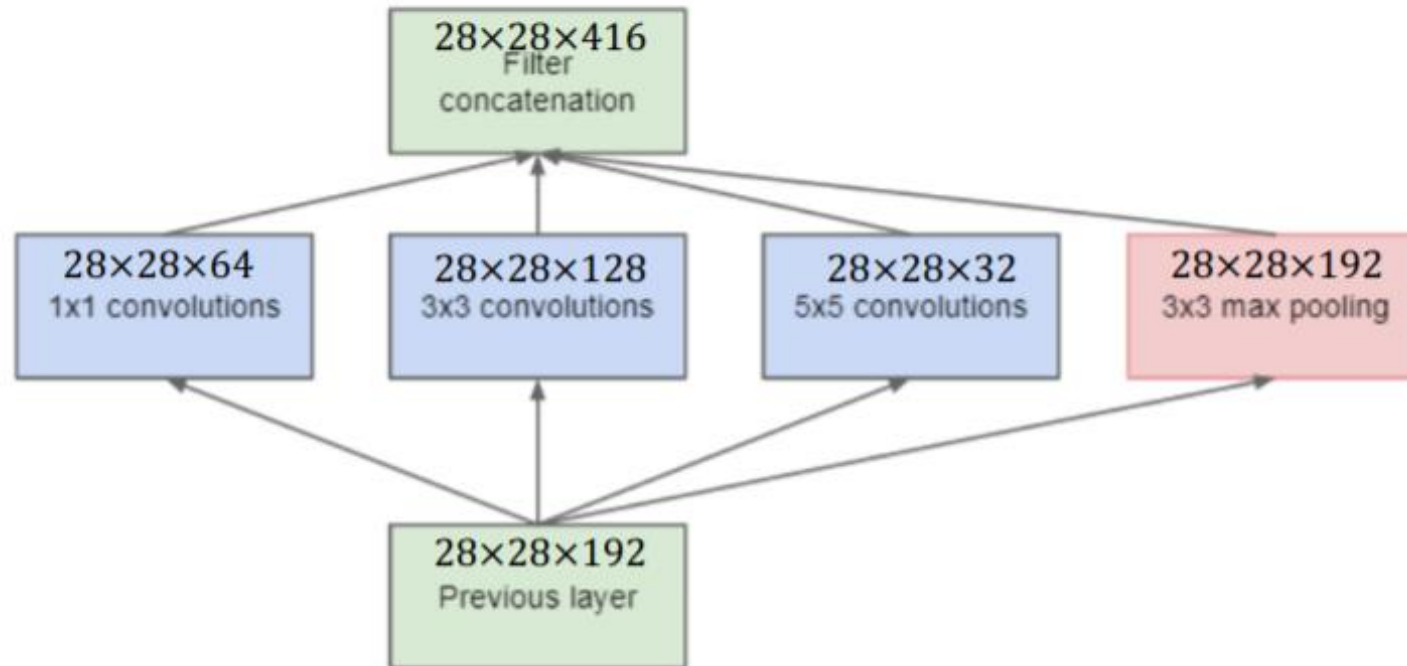


Inception Layer

$$28 \times 28 \times (64 + 128 + 32 + 192) = 28 \times 28 \times 416$$

What are the output sizes of all different filter operations?

What is output size after filter concatenation?

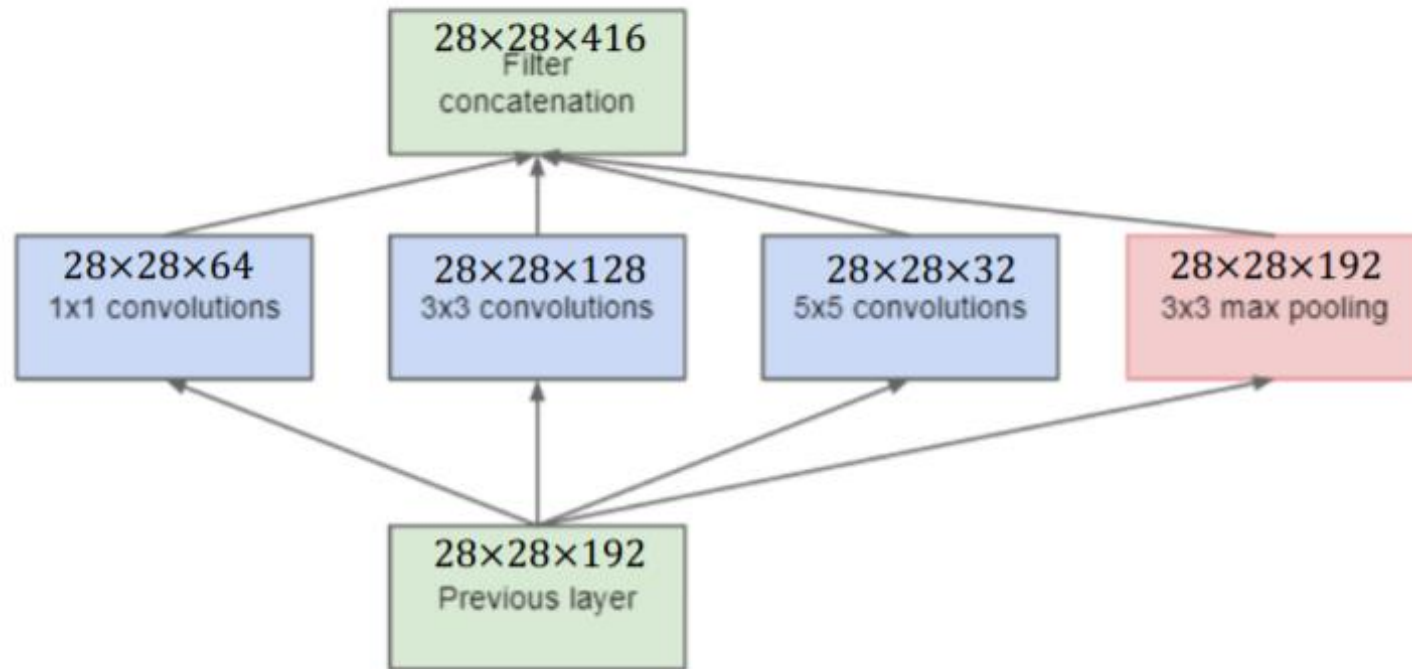


Inception Layer

Conv Ops:

[5x5 conv, 32]

$28 \times 28 \times 32 \times 5 \times 5 \times 192 = 120M$



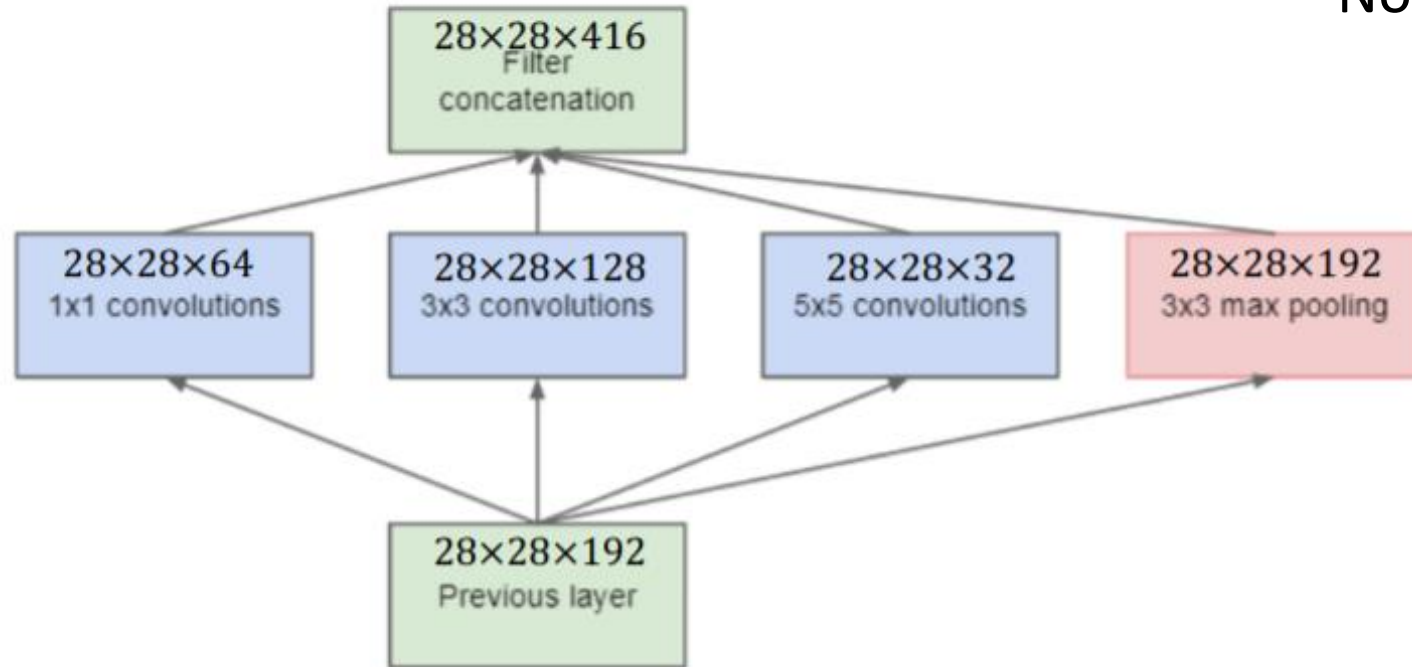
Inception Layer

Conv Ops:

[5x5 conv, 32]

$28 \times 28 \times 32 \times 5 \times 5 \times 192 = 120\text{M}$

Not sustainable



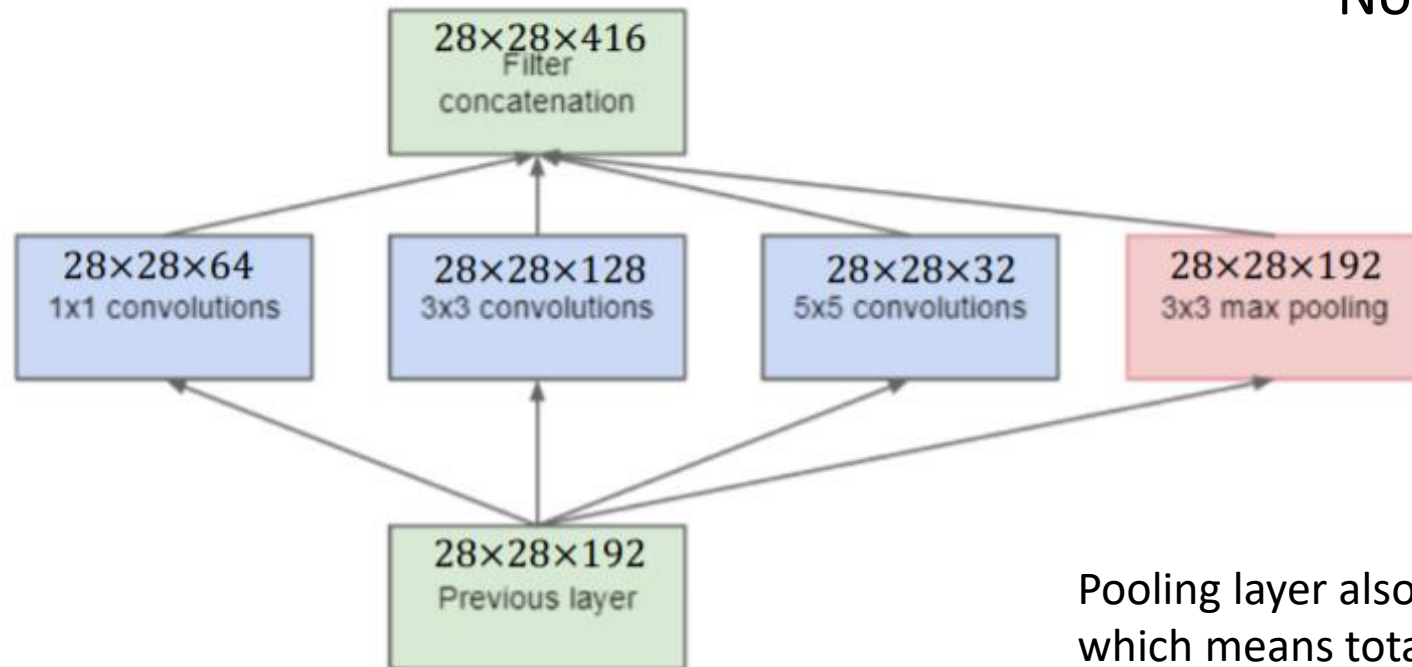
Inception Layer

Conv Ops:

[5x5 conv, 32]

$28 \times 28 \times 32 \times 5 \times 5 \times 192 = 120\text{M}$

Not sustainable



Pooling layer also preserves feature depth, which means total depth after concatenation can only grow at every layer!

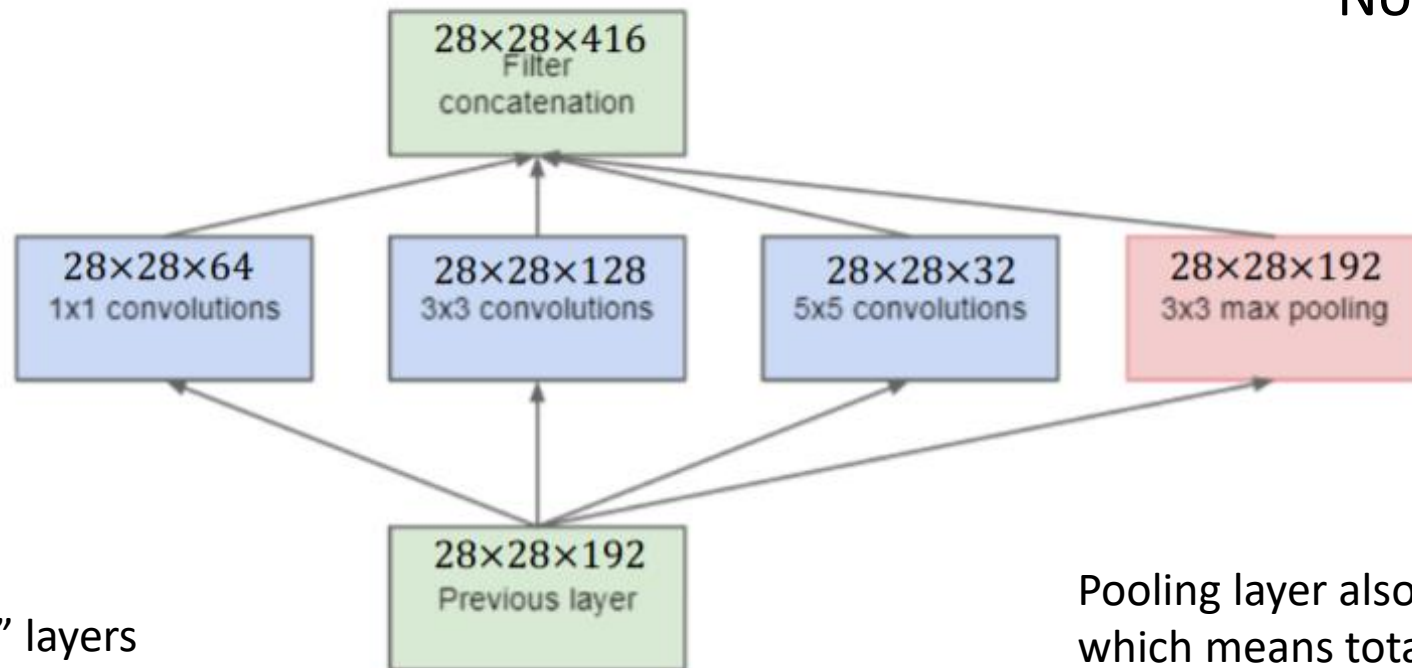
Inception Layer

Conv Ops:

[5x5 conv, 32]

$28 \times 28 \times 32 \times 5 \times 5 \times 192 = 120\text{M}$

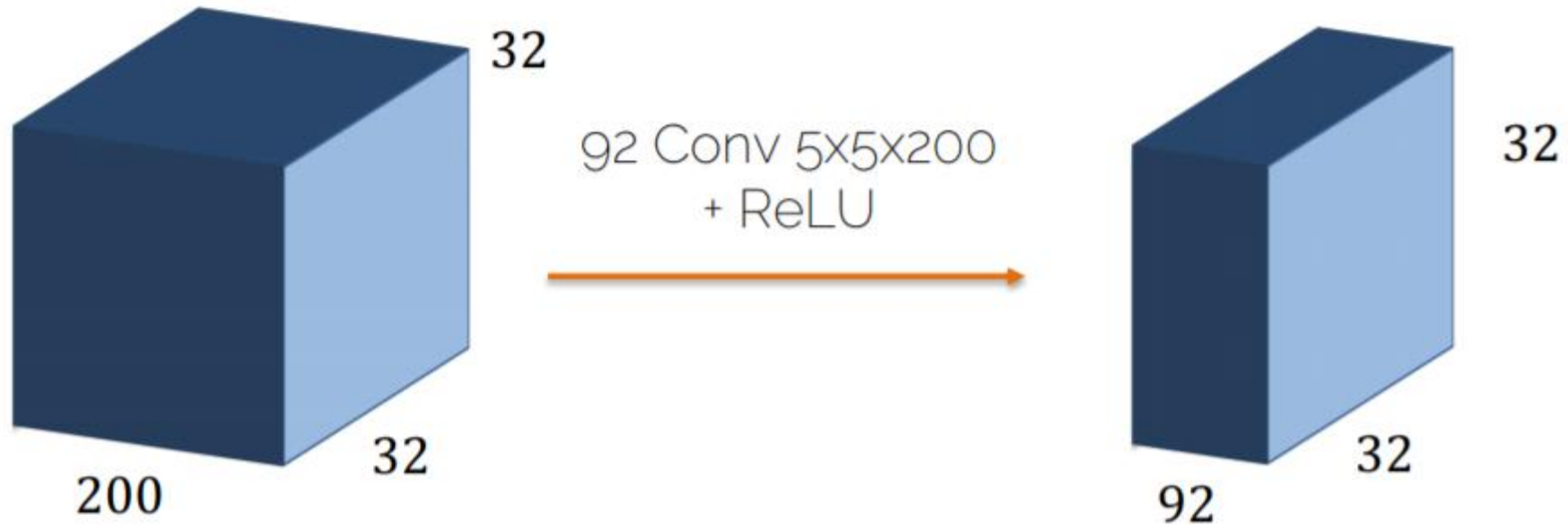
Not sustainable



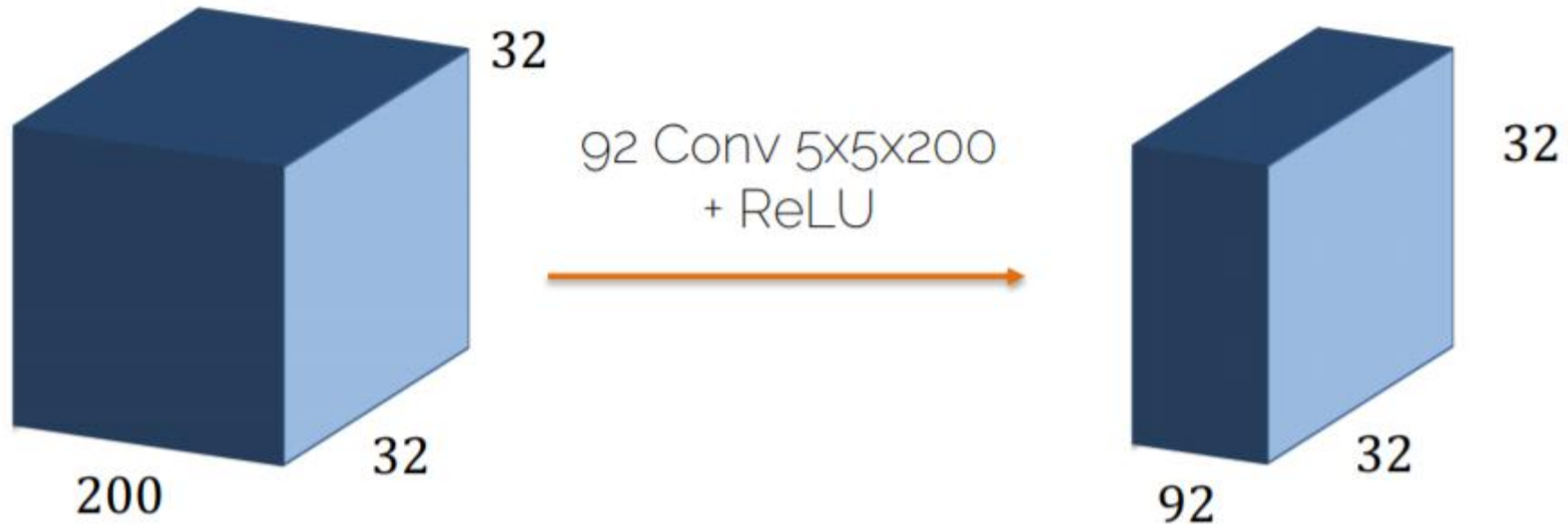
Solution: “bottleneck” layers that use 1x1 convolutions to reduce feature channel size

Pooling layer also preserves feature depth, which means total depth after concatenation can only grow at every layer!

Inception Layer: Computational Cost



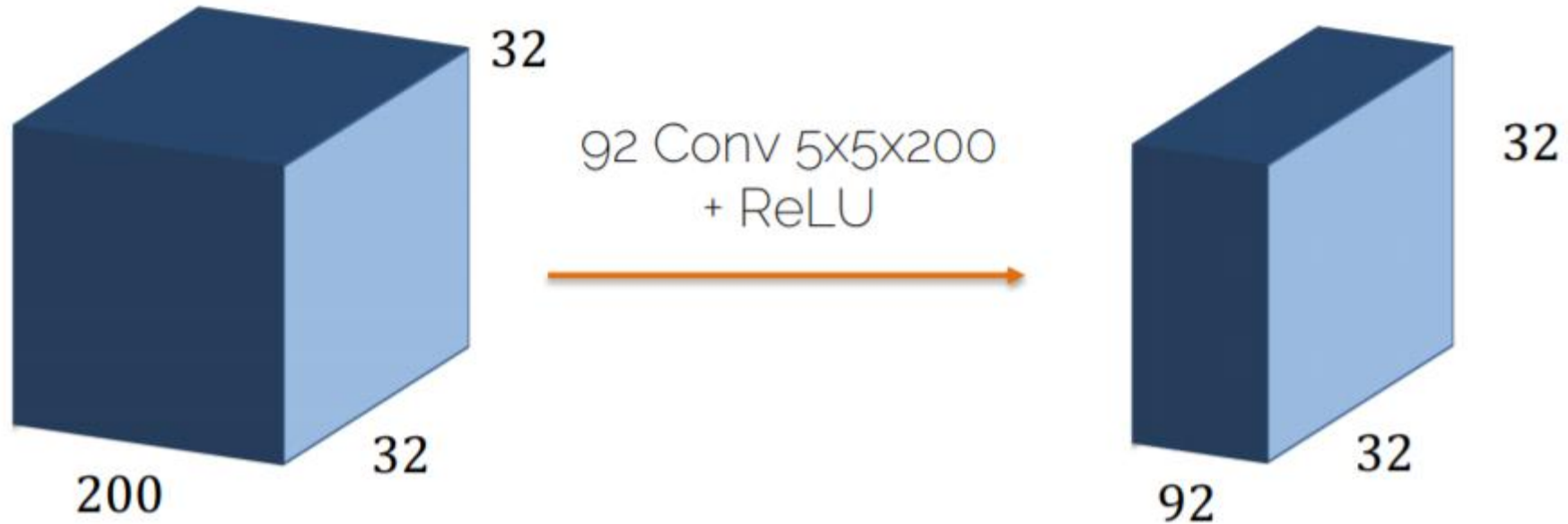
Inception Layer: Computational Cost



Multiplications: $5 \times 5 \times 200 \times 32 \times 32 \times 92$

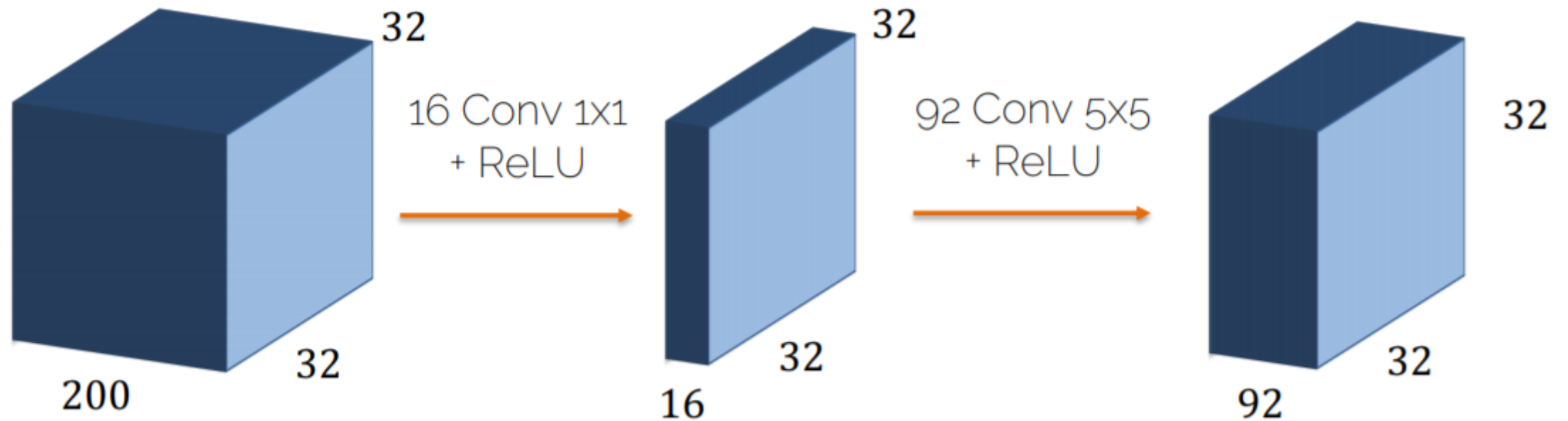
1 value of the output volume

Inception Layer: Computational Cost

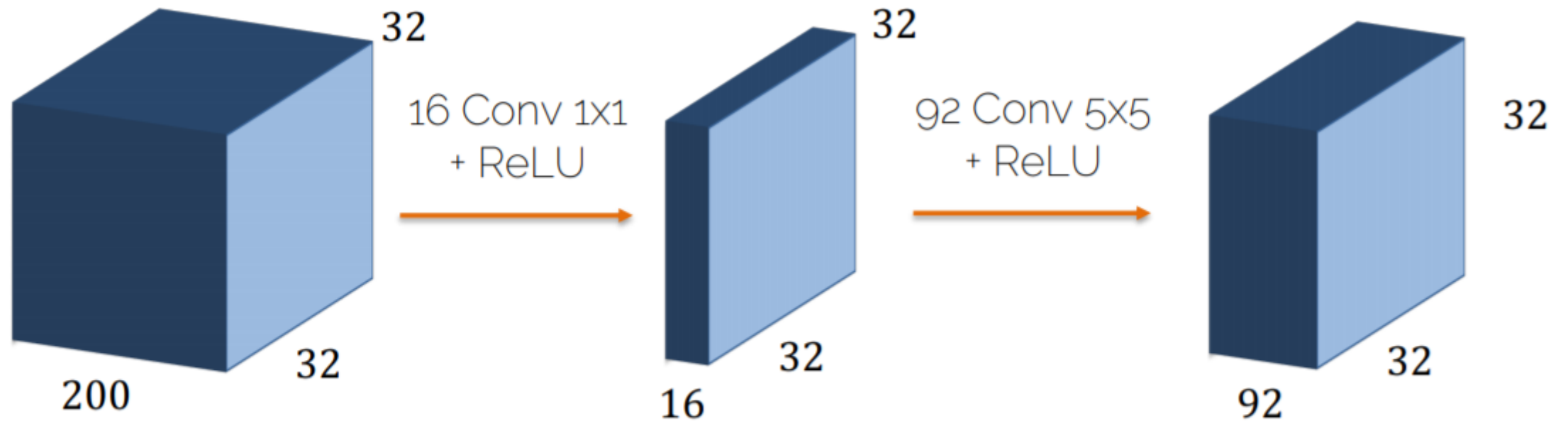


Multiplications: $5 \times 5 \times 200 \times 32 \times 32 \times 92 \sim 470 \text{ million}$

Inception Layer: Computational Cost

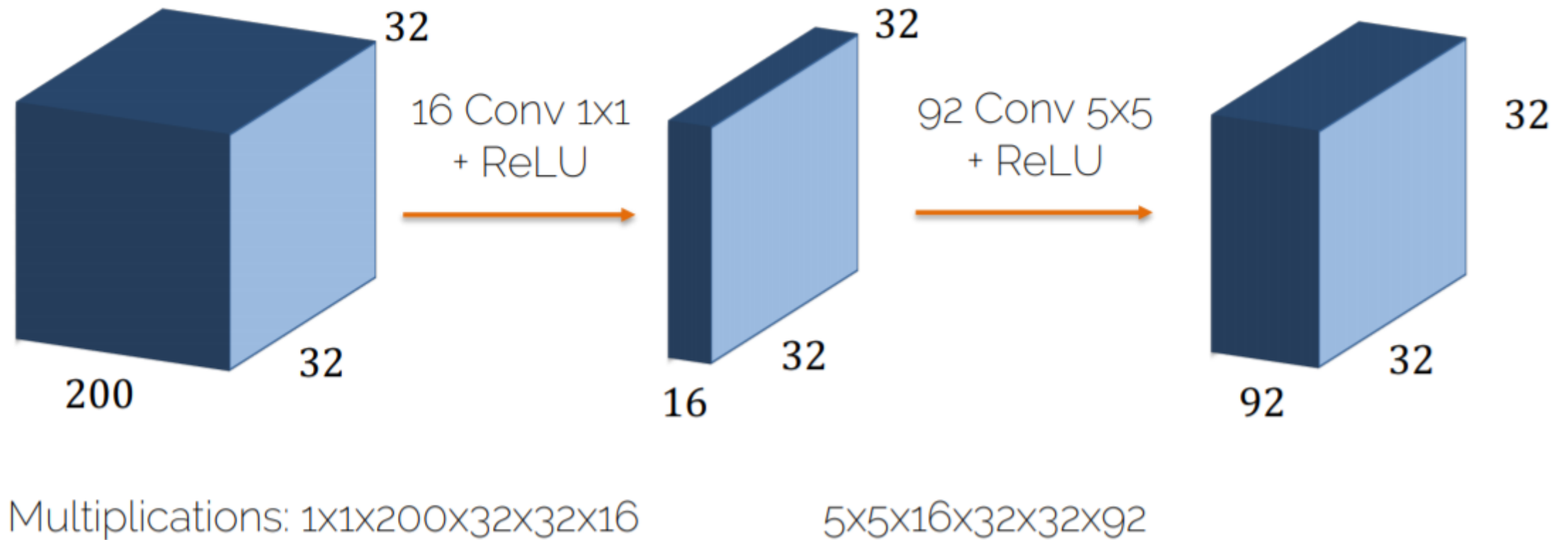


Inception Layer: Computational Cost

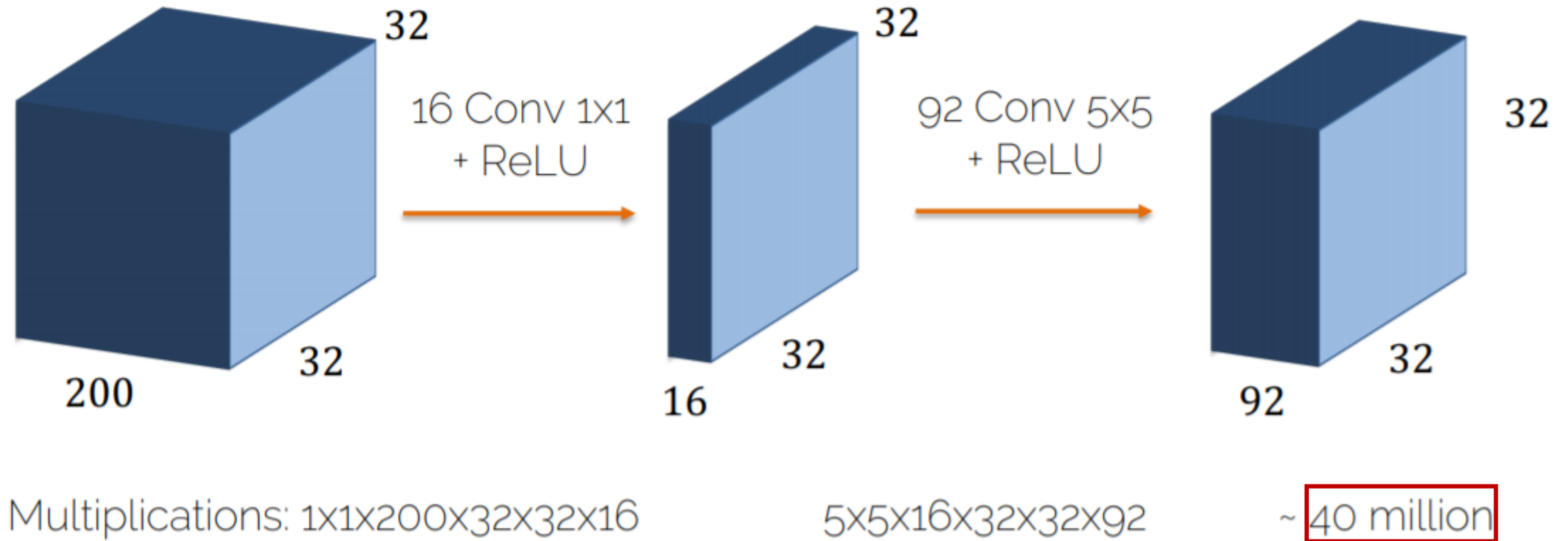


Multiplications: $1 \times 1 \times 200 \times 32 \times 32 \times 16$

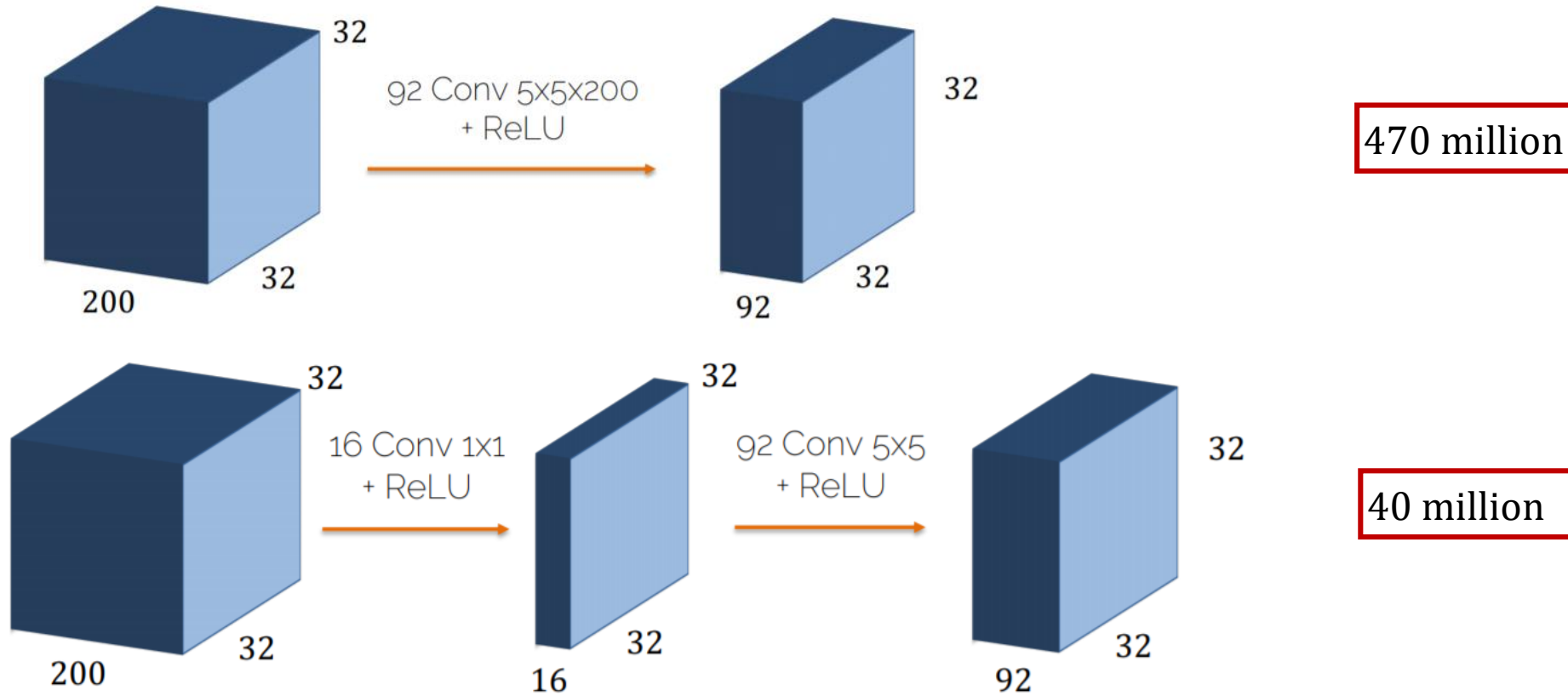
Inception Layer: Computational Cost



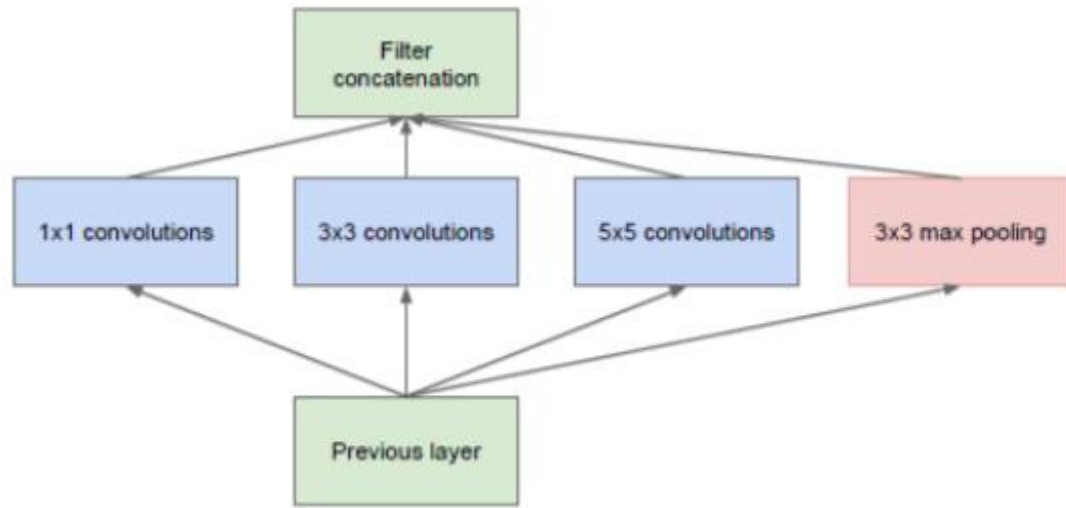
Inception Layer: Computational Cost



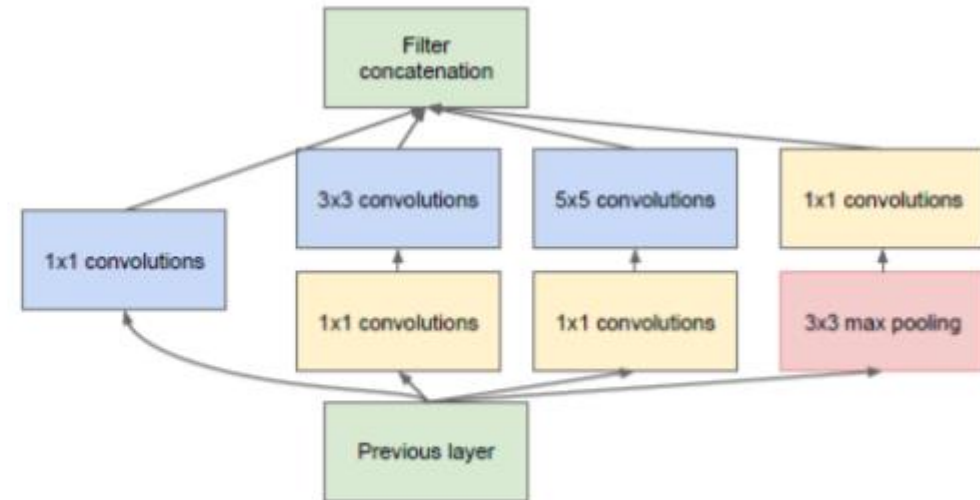
Inception Layer: Computational Cost



Inception Layer

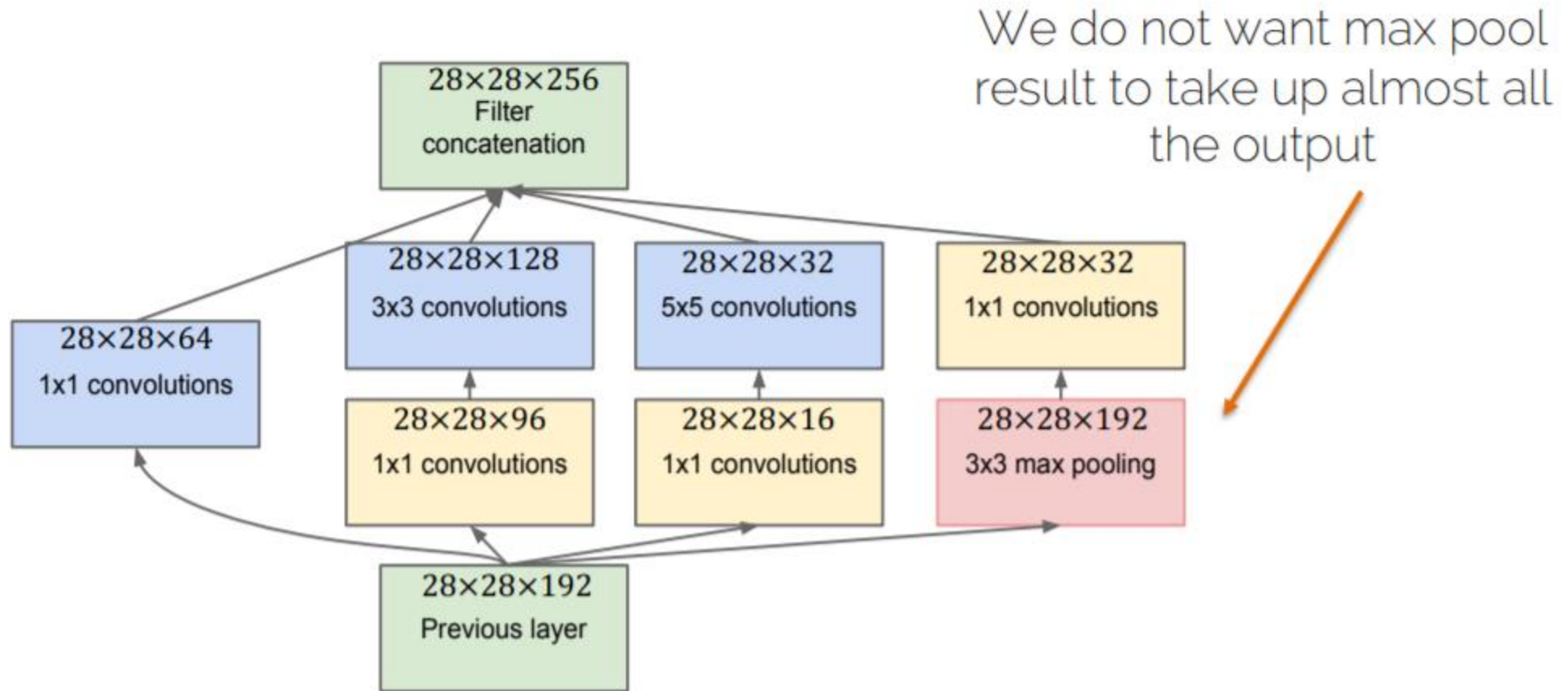


(a) Inception module, naïve version

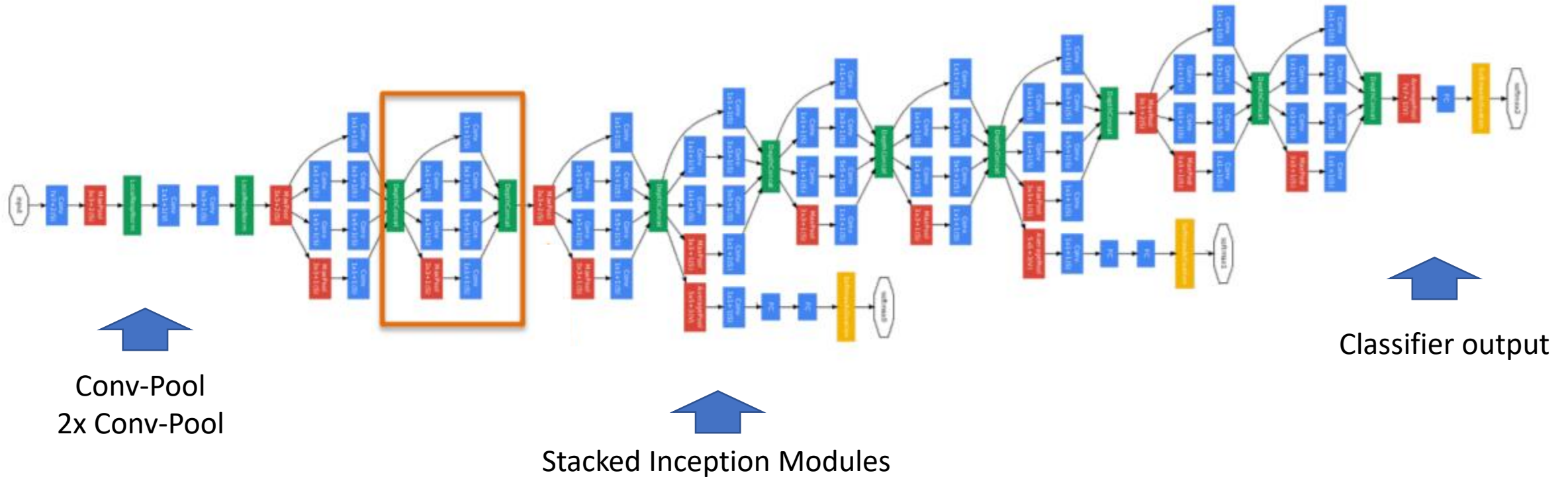


(b) Inception module with dimensionality reduction

Inception Layer



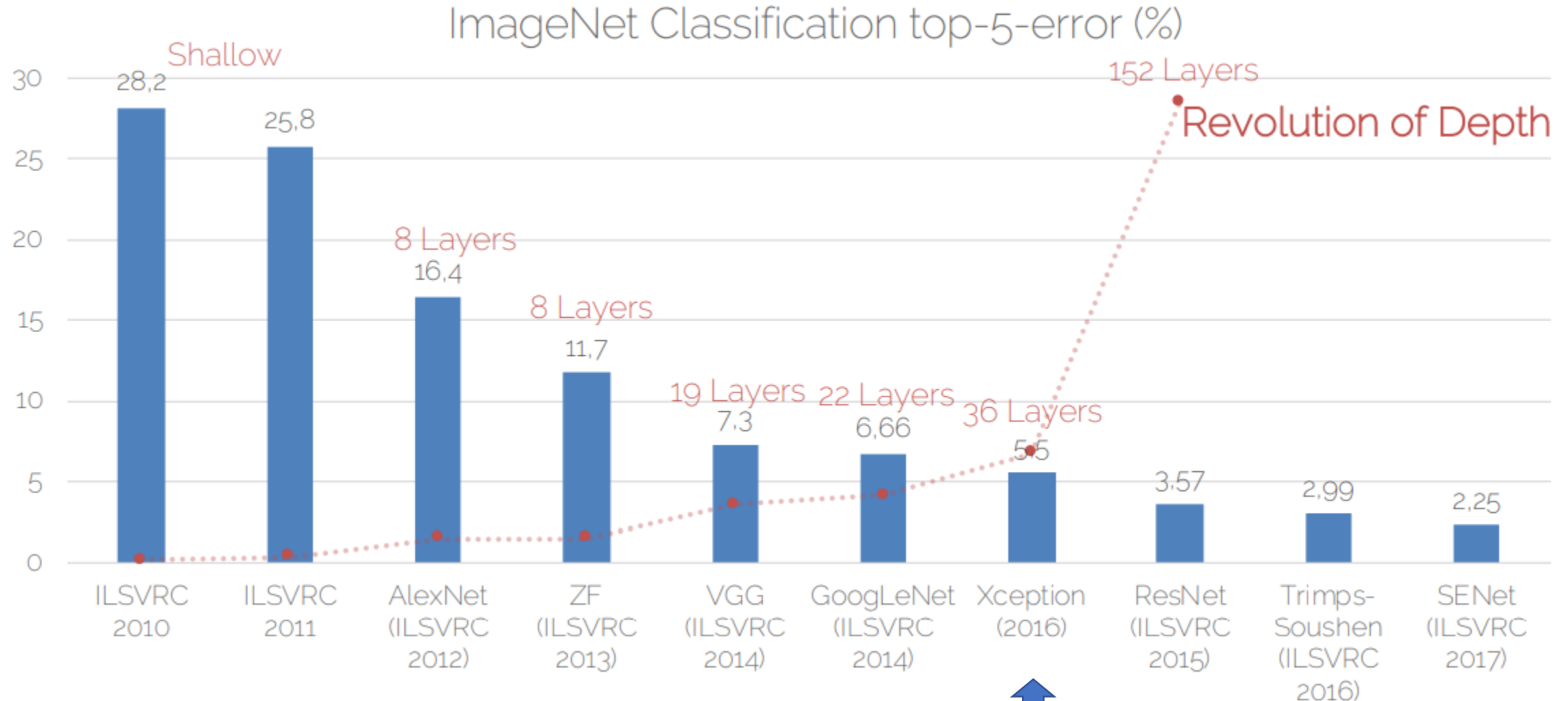
GoogLeNet



Auxiliary classification outputs to inject additional gradient at lower layers

[Szegedy et al CVPR'15] GoogLeNet

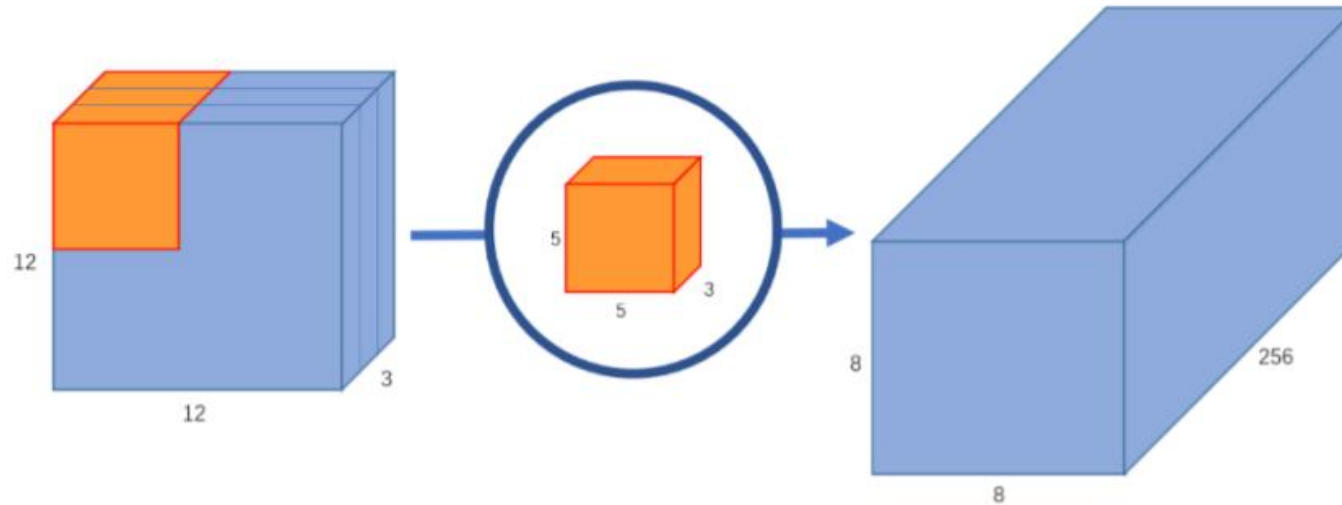
ImageNet Benchmark



Xception Net

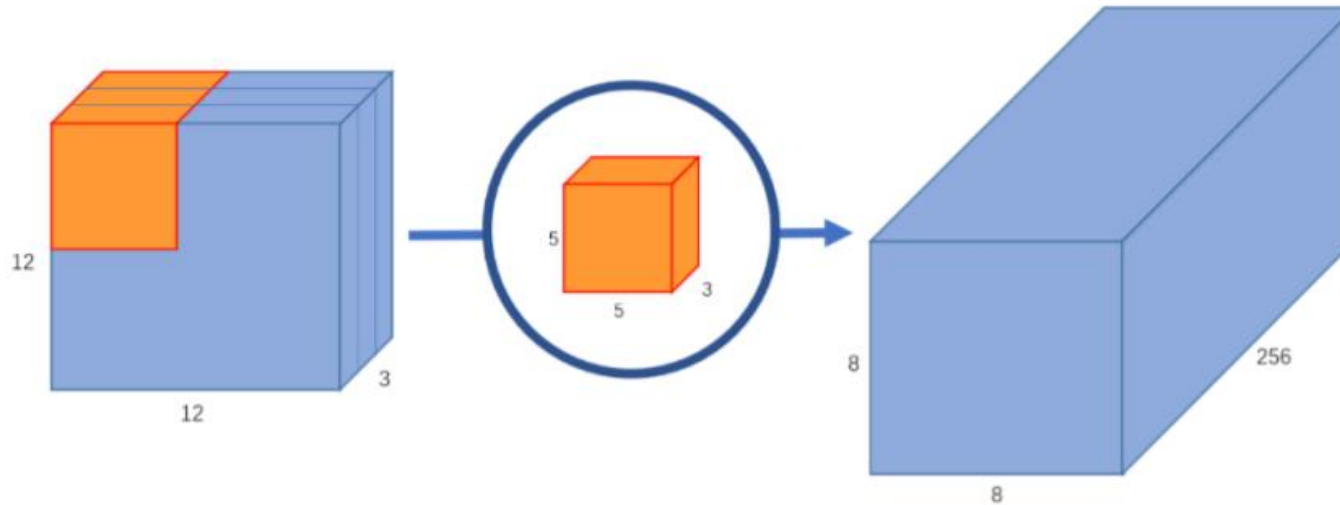
- Extreme version of Inception: applying (modified) **Depthwise Separable Convolutions** instead of normal convolutions
- 36 conv layers, structured into several modules with skip connections
- Outperforms Inception Net V3

Depth-wise separable convolutions

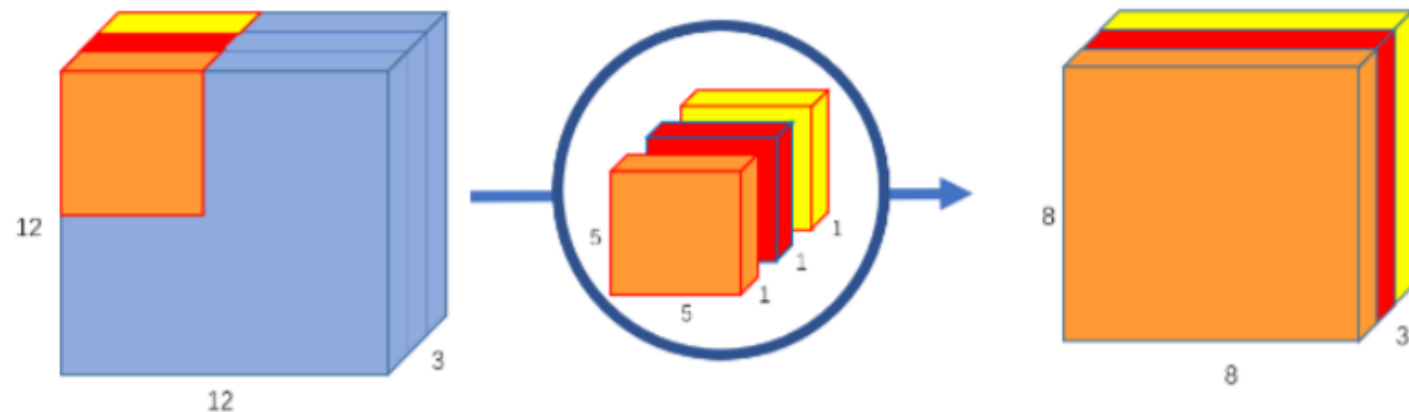


Normal convolutions act on all channels.

Depth-wise separable convolutions

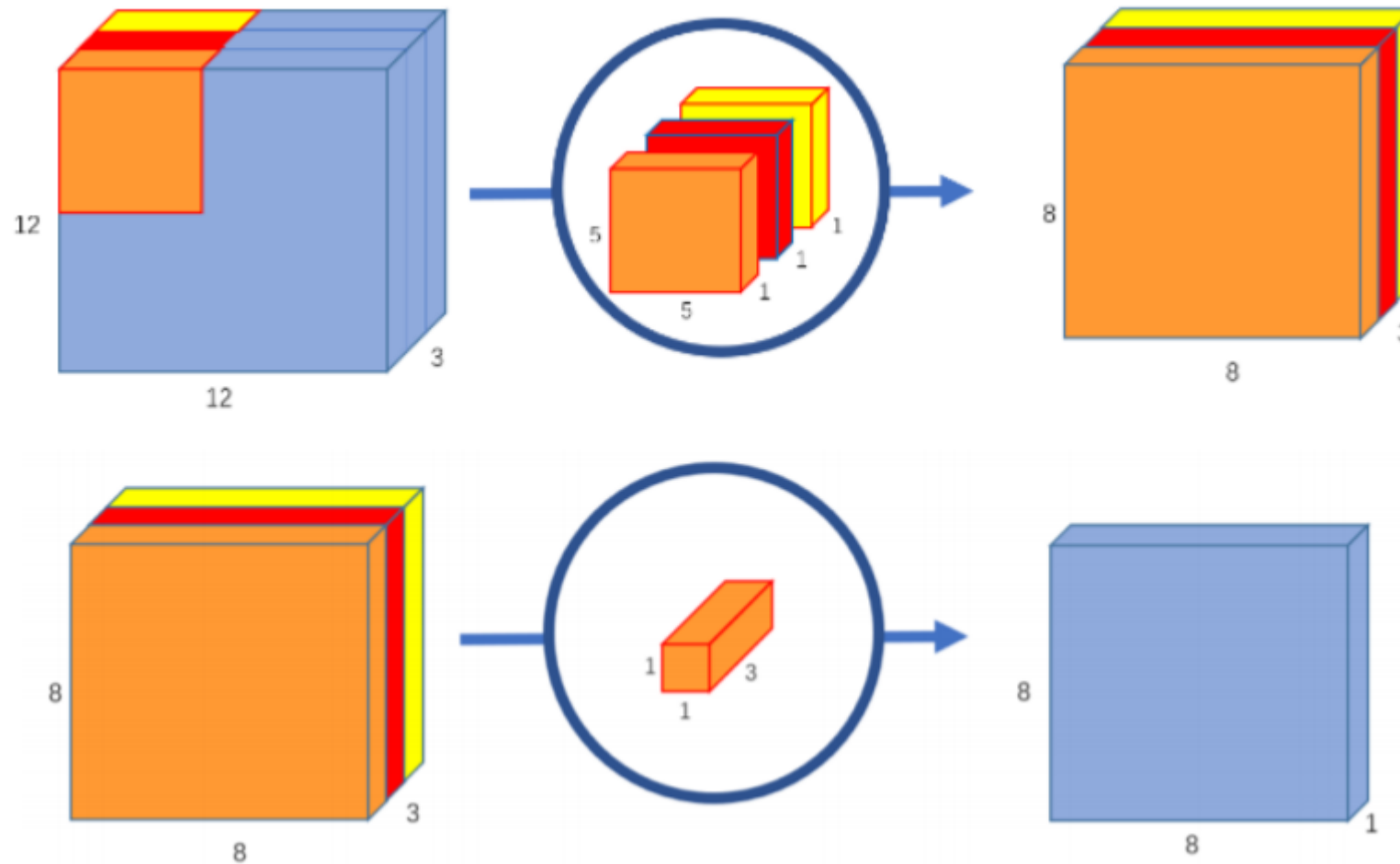


Normal convolutions act on all channels.



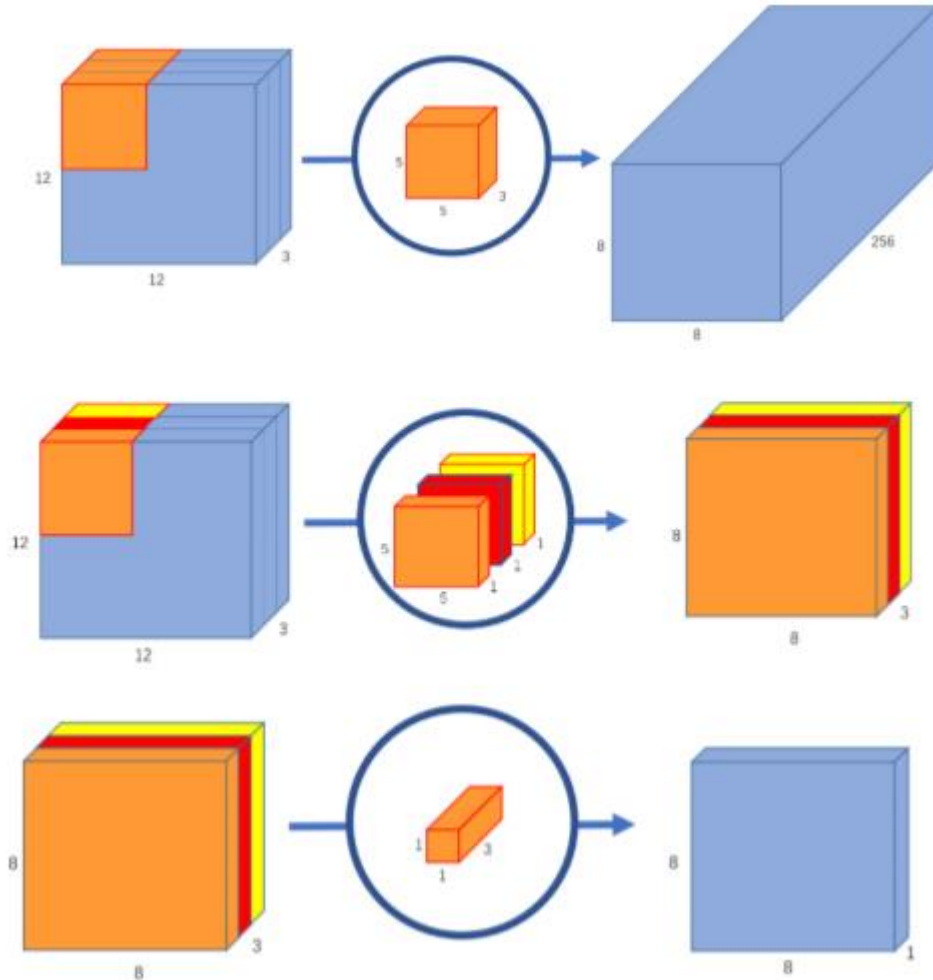
Filters are applied only at certain depths of the features.

Depth-wise separable convolutions



1x1 convs

Computational Cost



Original convolution

256 kernels of size 5x5x3

Multiplications:

$$256 \times 5 \times 5 \times 3 \times (8 \times 8 \text{ locations}) = 1.228.800$$

Depth-wise convolution

3 kernels of size 5x5x1

Multiplications:

$$5 \times 5 \times 3 \times (8 \times 8 \text{ locations}) = 4800$$

1x1 convolution

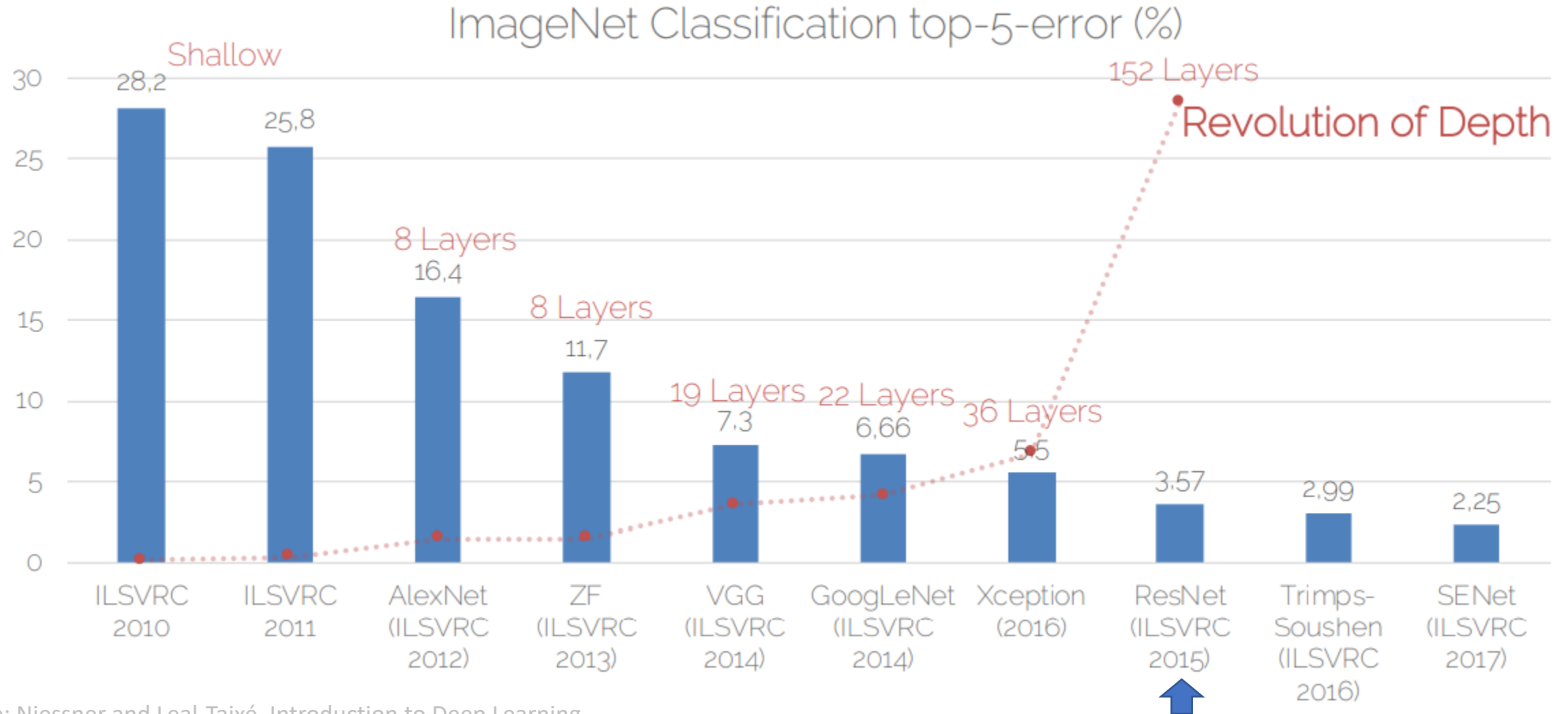
256 kernels of size 1x1x3

Multiplications:

$$256 \times 1 \times 1 \times 3 \times (8 \times 8 \text{ locations}) = 49152$$

Less
computations!

ImageNet Benchmark



The Problem of Depth

What happens when we continue stacking deeper layers on a “plain” convolutional neural network?

The Problem of Depth

What happens when we continue stacking deeper layers on a “plain” convolutional neural network?



The Problem of Depth

What happens when we continue stacking deeper layers on a “plain” convolutional neural network?

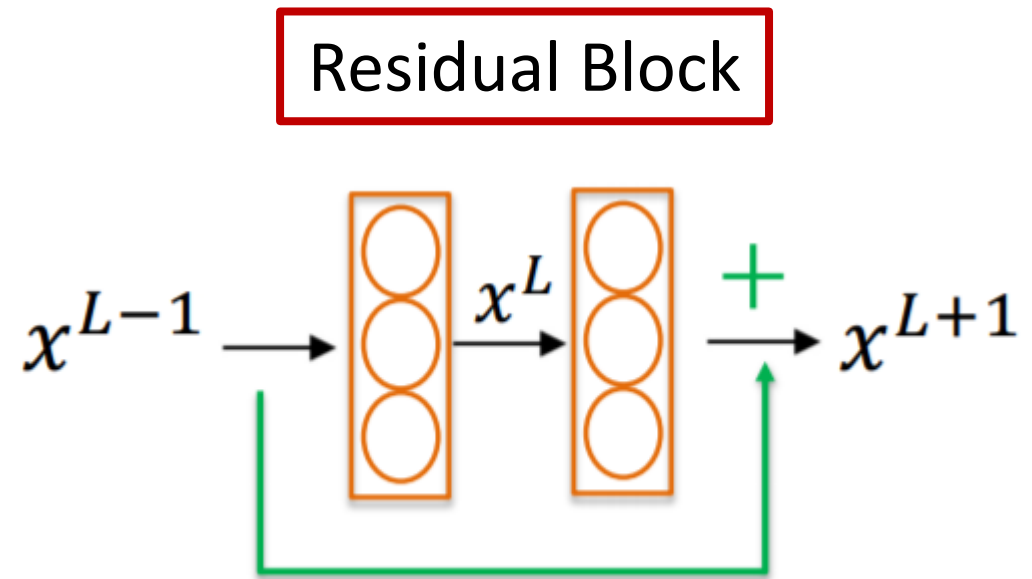


56-layer model performs worse on both training and test error

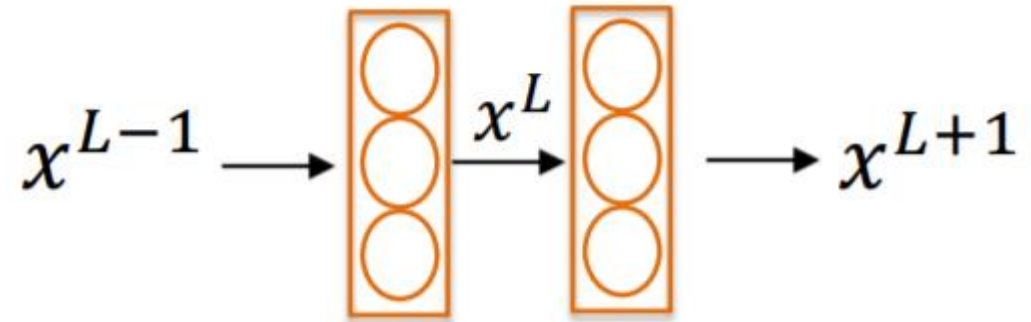
-> The deeper model performs worse, **but it's not caused by overfitting**

The Problem of Depth

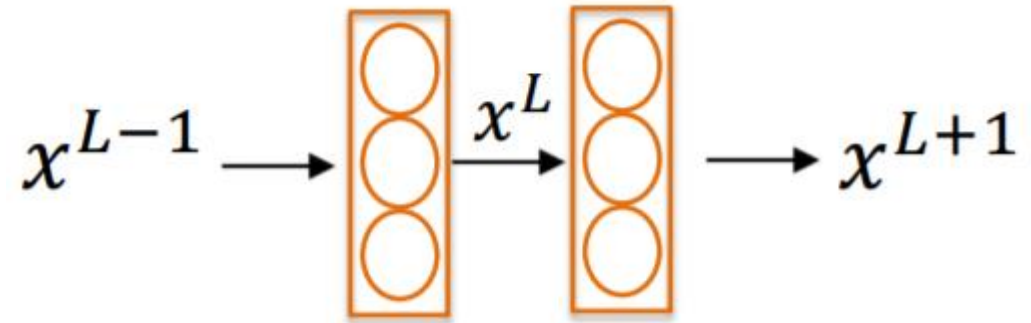
- As we add more and more layers, training becomes harder
- Vanishing and exploding gradients
- How can we train very deep nets?



Residual Block

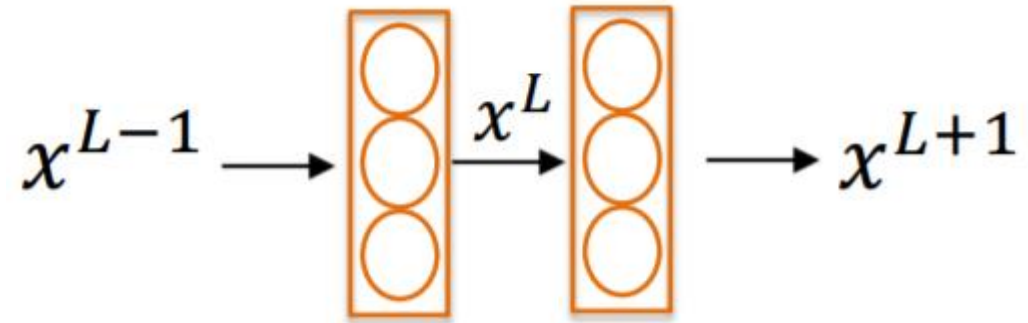


Residual Block



Input $\longrightarrow W^L x^{L-1} + b^L$

Residual Block

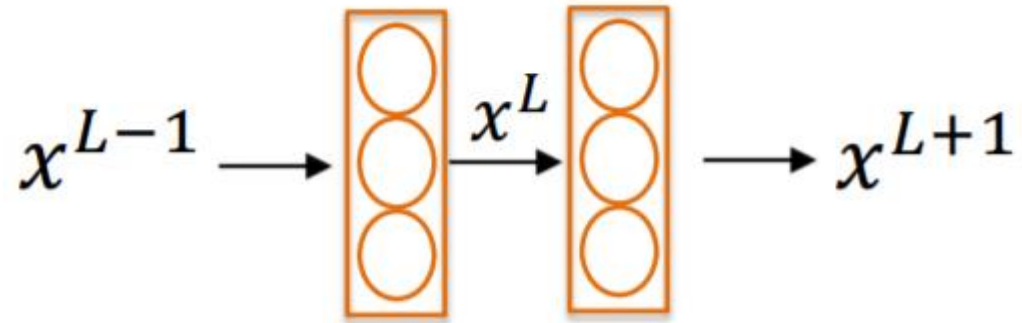


Input $\longrightarrow W^L x^{L-1} + b^L \longrightarrow x^L$

Linear

A small graph showing a linear relationship with a positive slope, illustrating the linear transformation applied in the residual block.

Residual Block

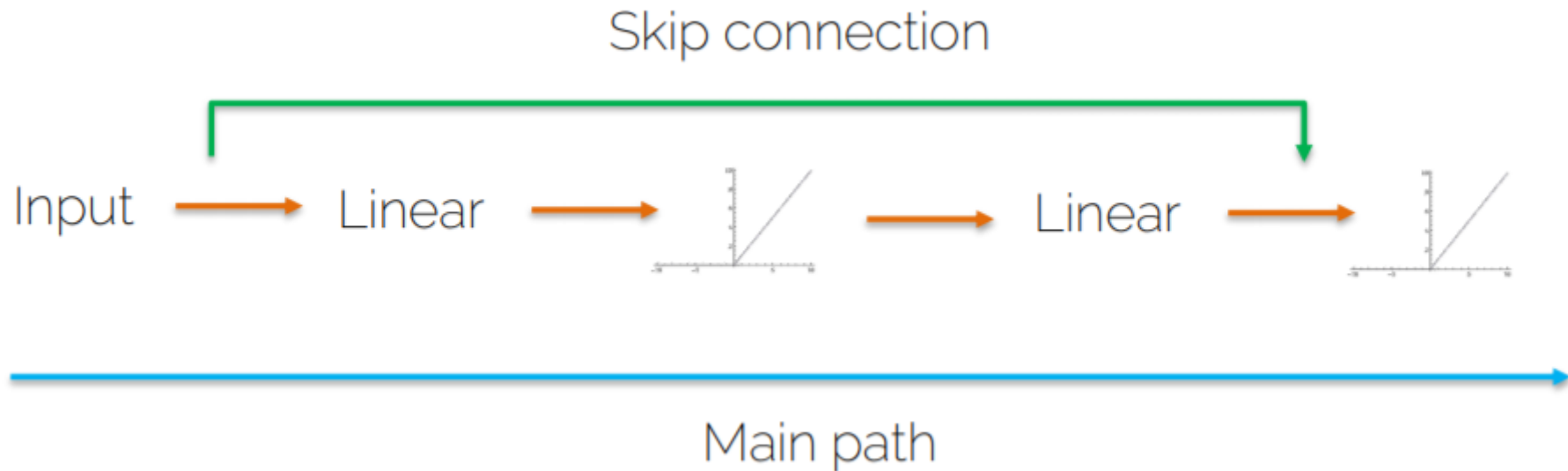
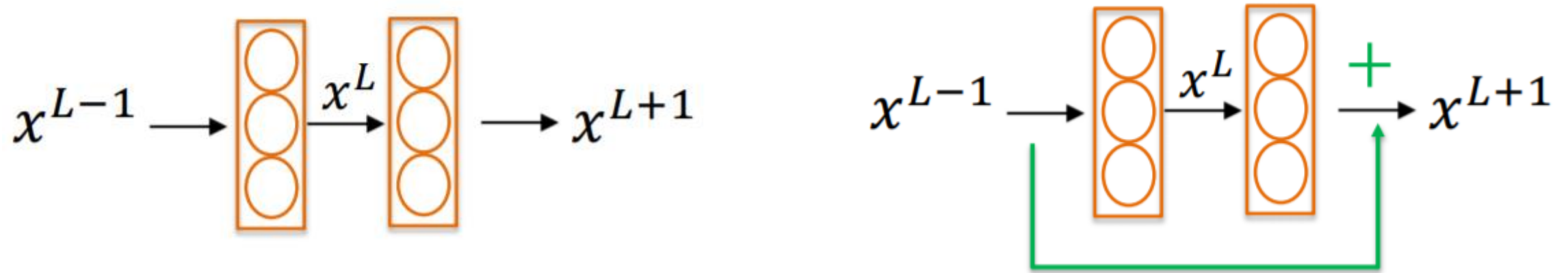


Input $\longrightarrow W^L x^{L-1} + b^L \xrightarrow{\text{Linear}} x^L = f(W^L x^{L-1} + b^L) \xrightarrow{\text{Non-linearity}}$

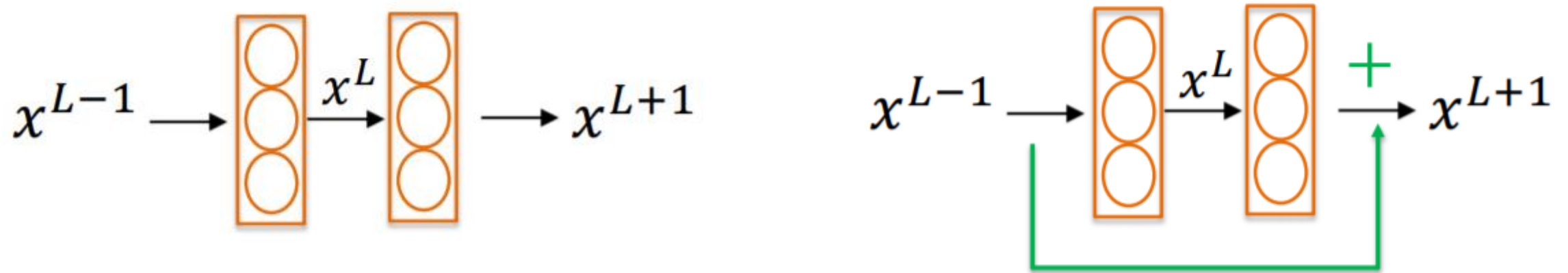


$\longrightarrow x^{L+1} = f(W^{L+1} x^L + b^{L+1})$

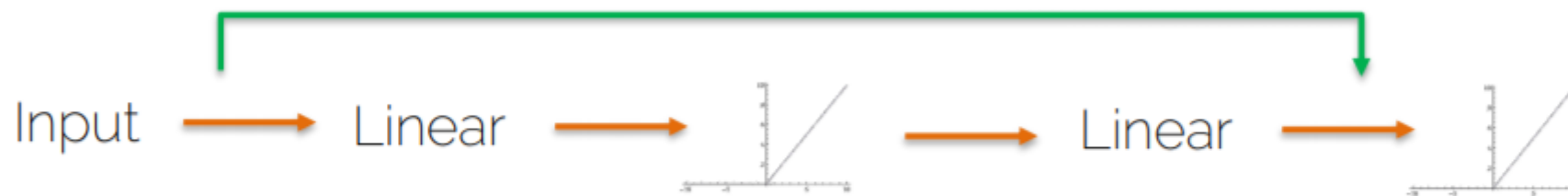
Residual Block



Residual Block



$$x^{L+1} = f(W^{L+1}x^L + b^{L+1} + x^{L-1})$$

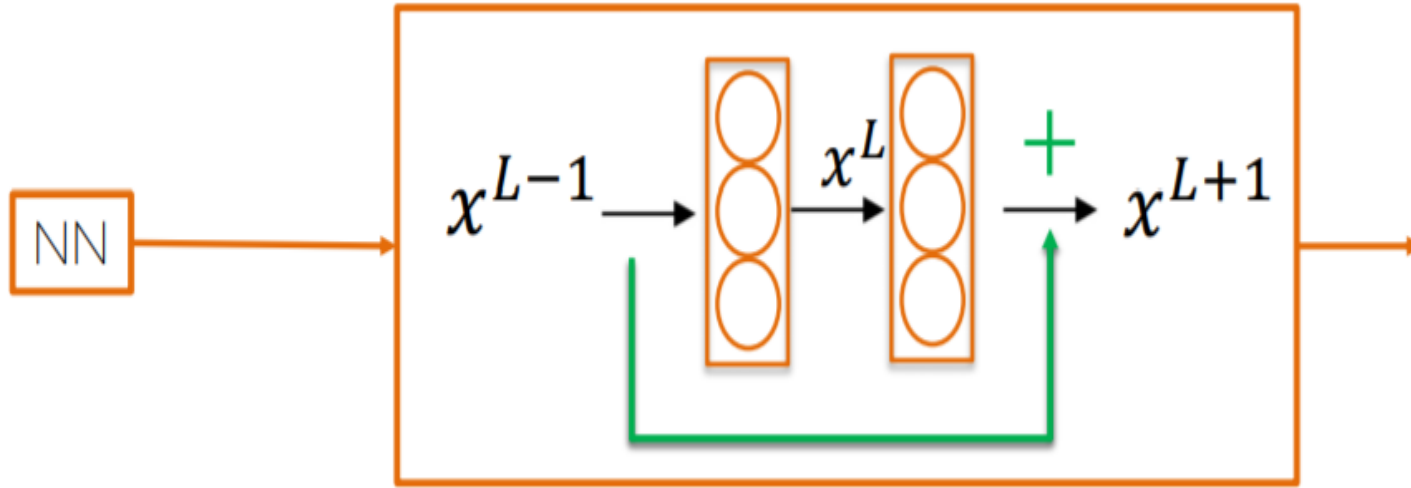


$$x^{L+1} = f(W^{L+1}x^L + b^{L+1})$$

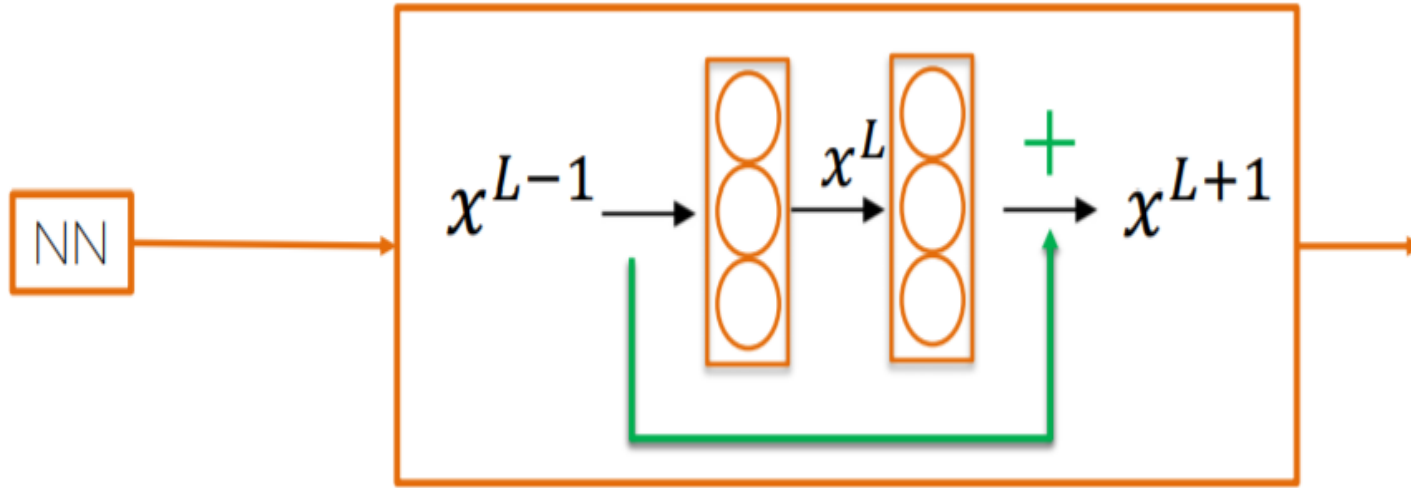
Why do ResNets Work?



Why do ResNets Work?

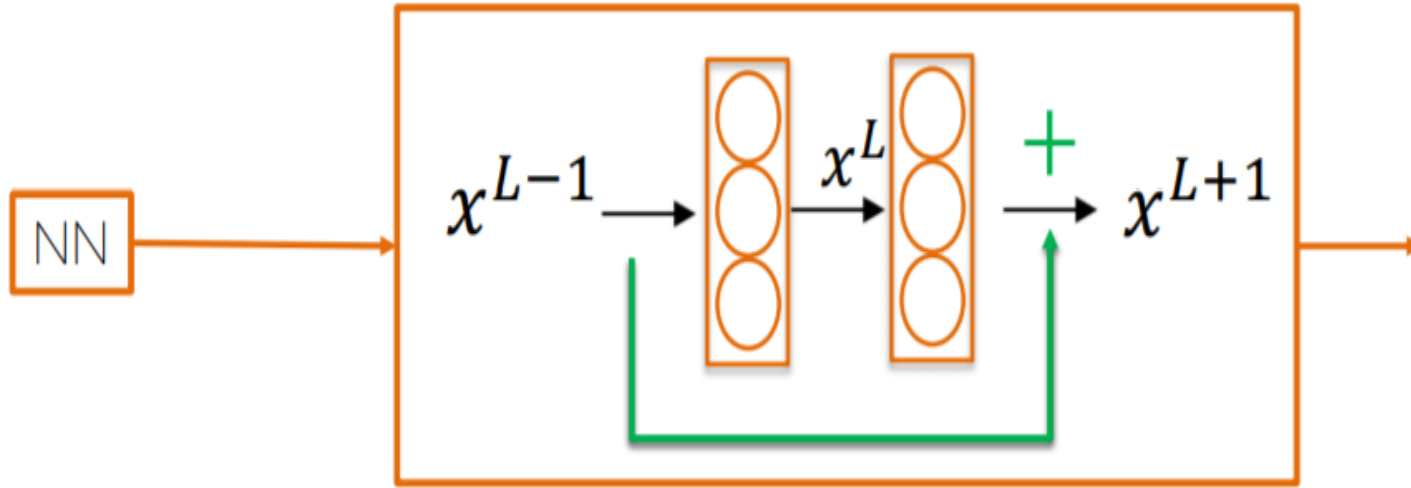


Why do ResNets Work?



$$x^{L+1} = f(W^{L+1}x^L + b^{L+1} + x^{L-1})$$

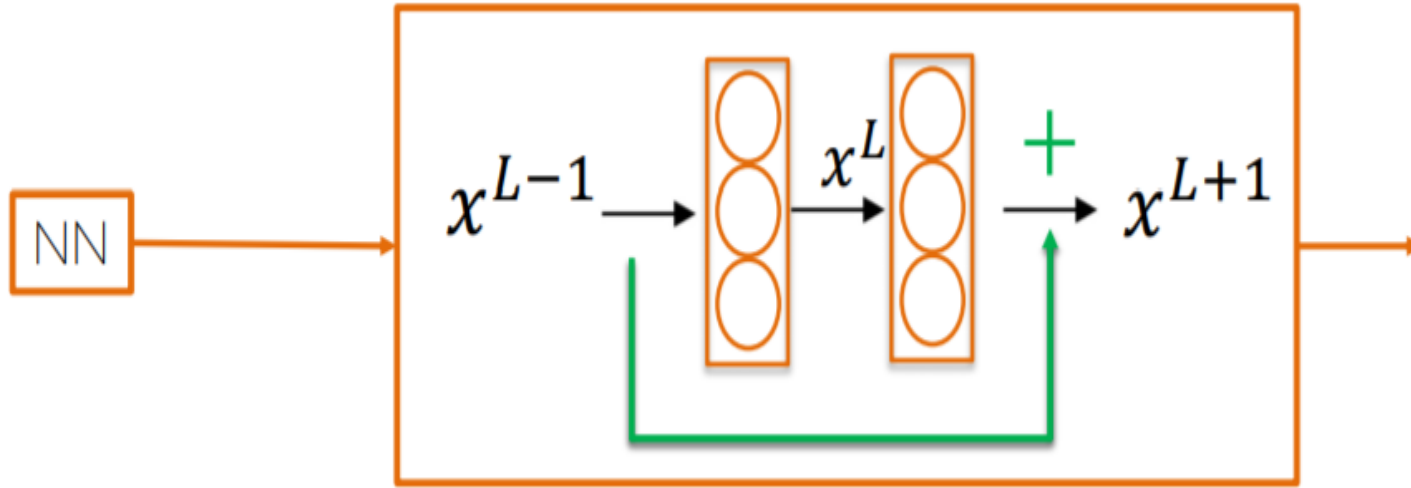
Why do ResNets Work?



$$x^{L+1} = f(\underbrace{W^{L+1}}_{\sim \text{zero}} x^L + \underbrace{b^{L+1}}_{\sim \text{zero}} + x^{L-1})$$

$$x^{L+1} = f(x^{L-1})$$

Why do ResNets Work?

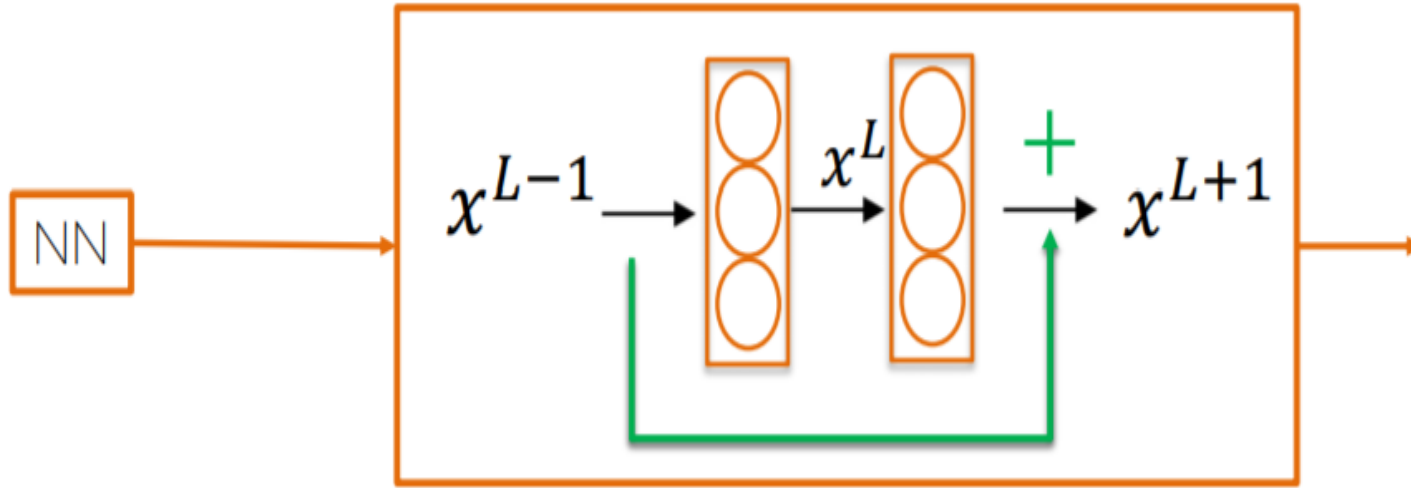


$$x^{L+1} = f(\underbrace{W^{L+1}}_{\sim \text{zero}} x^L + \underbrace{b^{L+1}}_{\sim \text{zero}} + x^{L-1})$$

$$x^{L+1} = f(x^{L-1})$$

We kept the same values and added a non-linearity

Why do ResNets Work?



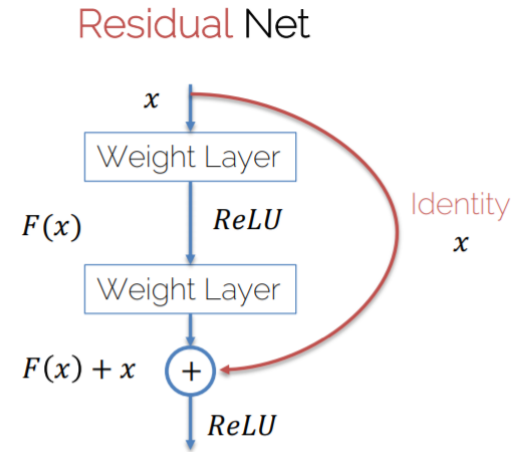
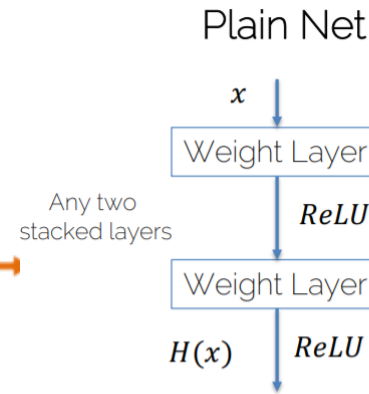
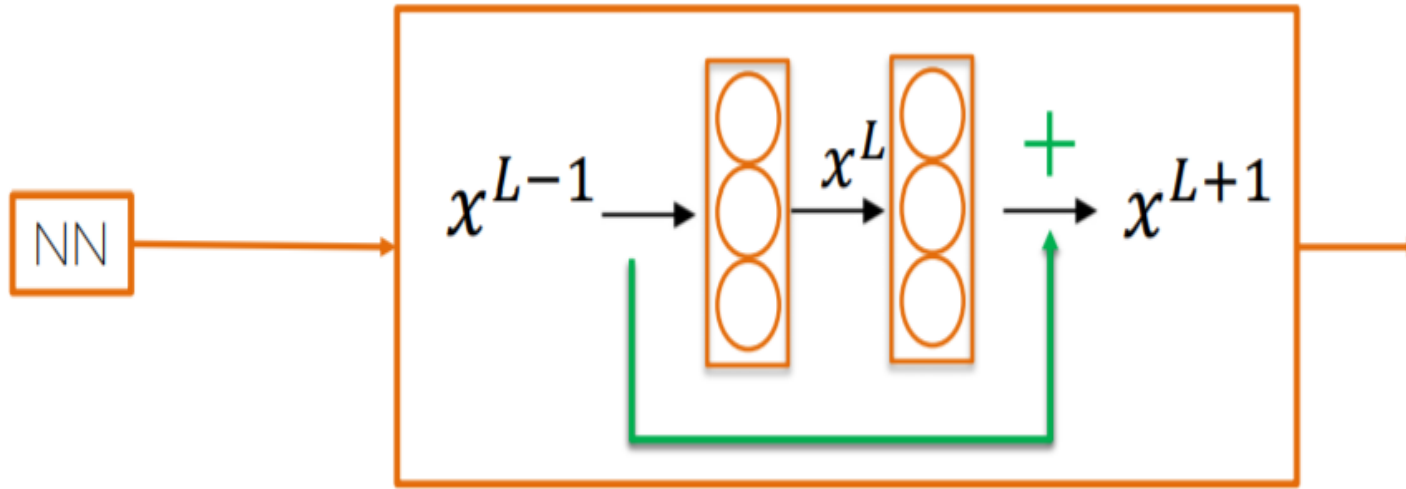
$$x^{L+1} = f(\underbrace{W^{L+1}}_{\sim \text{zero}} x^L + \underbrace{b^{L+1}}_{\sim \text{zero}} + x^{L-1})$$

$$x^{L+1} = f(x^{L-1})$$

We kept the same values and added a non-linearity

- The identity is easy for the residual block to learn
- Guaranteed it will not hurt

Why do ResNets Work?



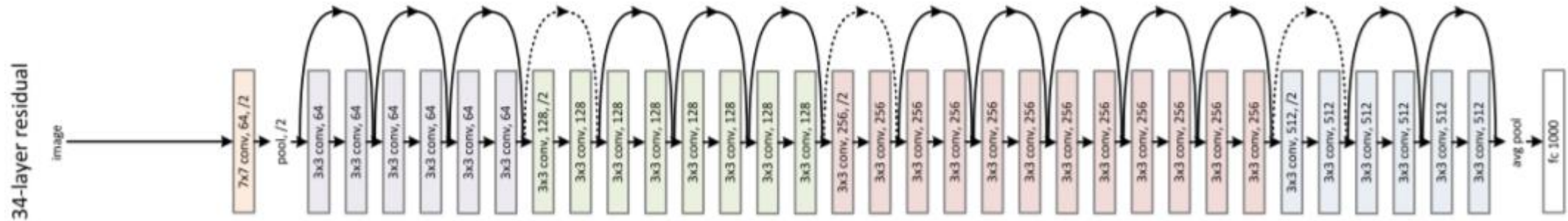
$$x^{L+1} = f(\underbrace{W^{L+1}x^L}_{\sim \text{zero}} + \underbrace{b^{L+1}}_{\sim \text{zero}} + x^{L-1})$$

$$x^{L+1} = f(x^{L-1})$$

We kept the same values and added a non-linearity

- The identity is easy for the residual block to learn
- Guaranteed it will not hurt

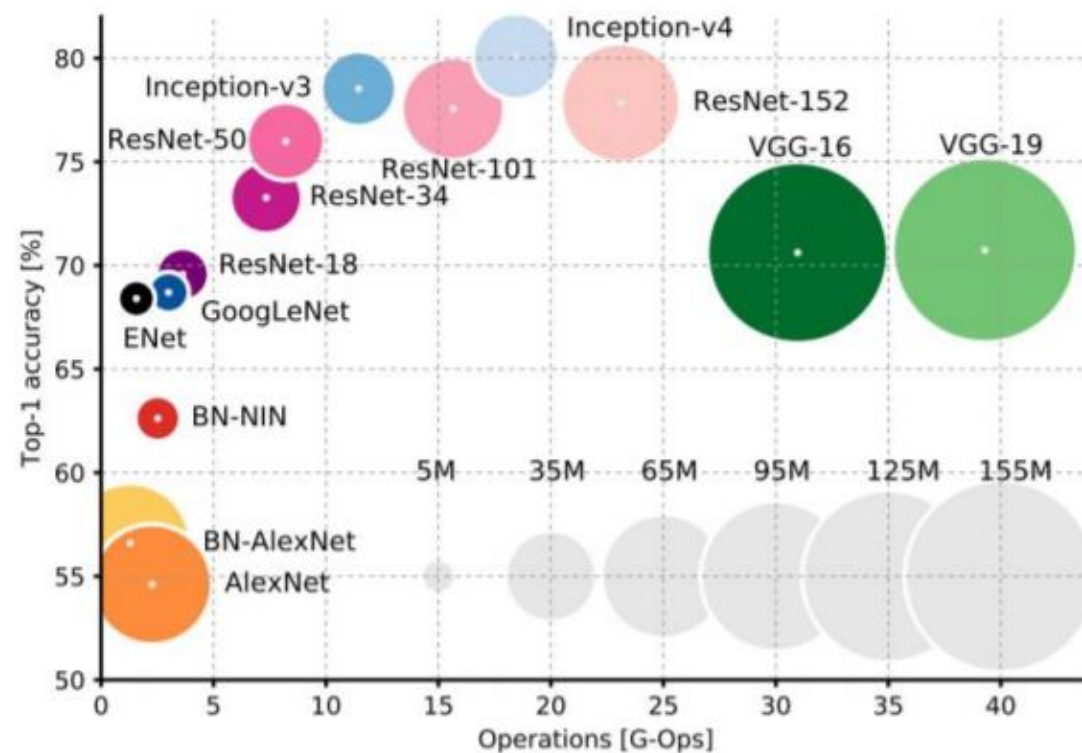
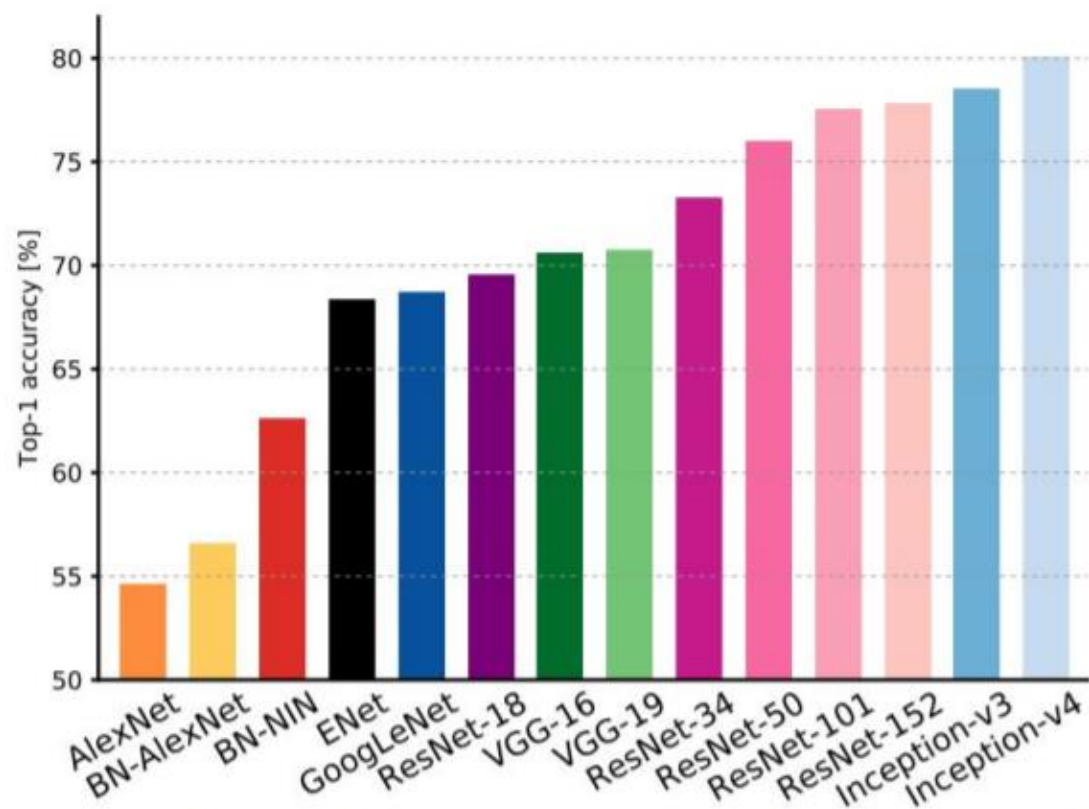
ResNet



- Xavier/2 initialization
- SGD + Momentum (0.9)
- Learning rate 0.1, divided by 10 when plateau
- Mini-batch size 256
- Weight decay of $1e-5$
- No dropout

ResNet-152: 60M parameters

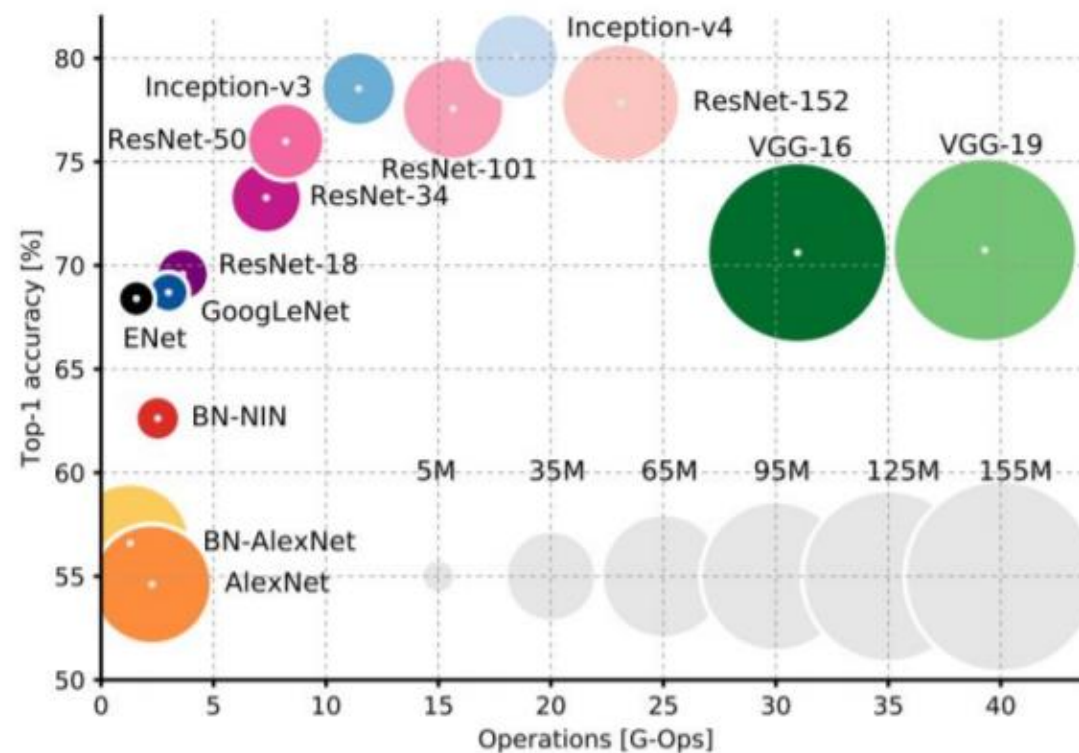
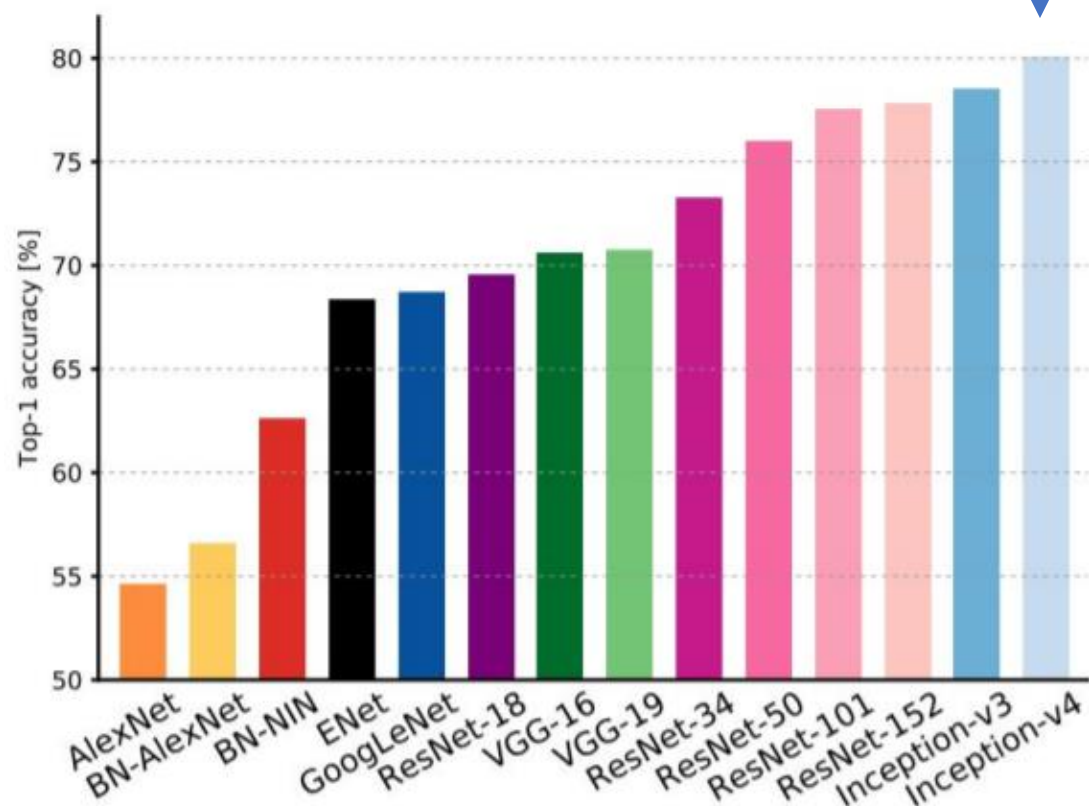
Comparing complexity



An Analysis of Deep Neural Network Models for Practical Applications, 2017.

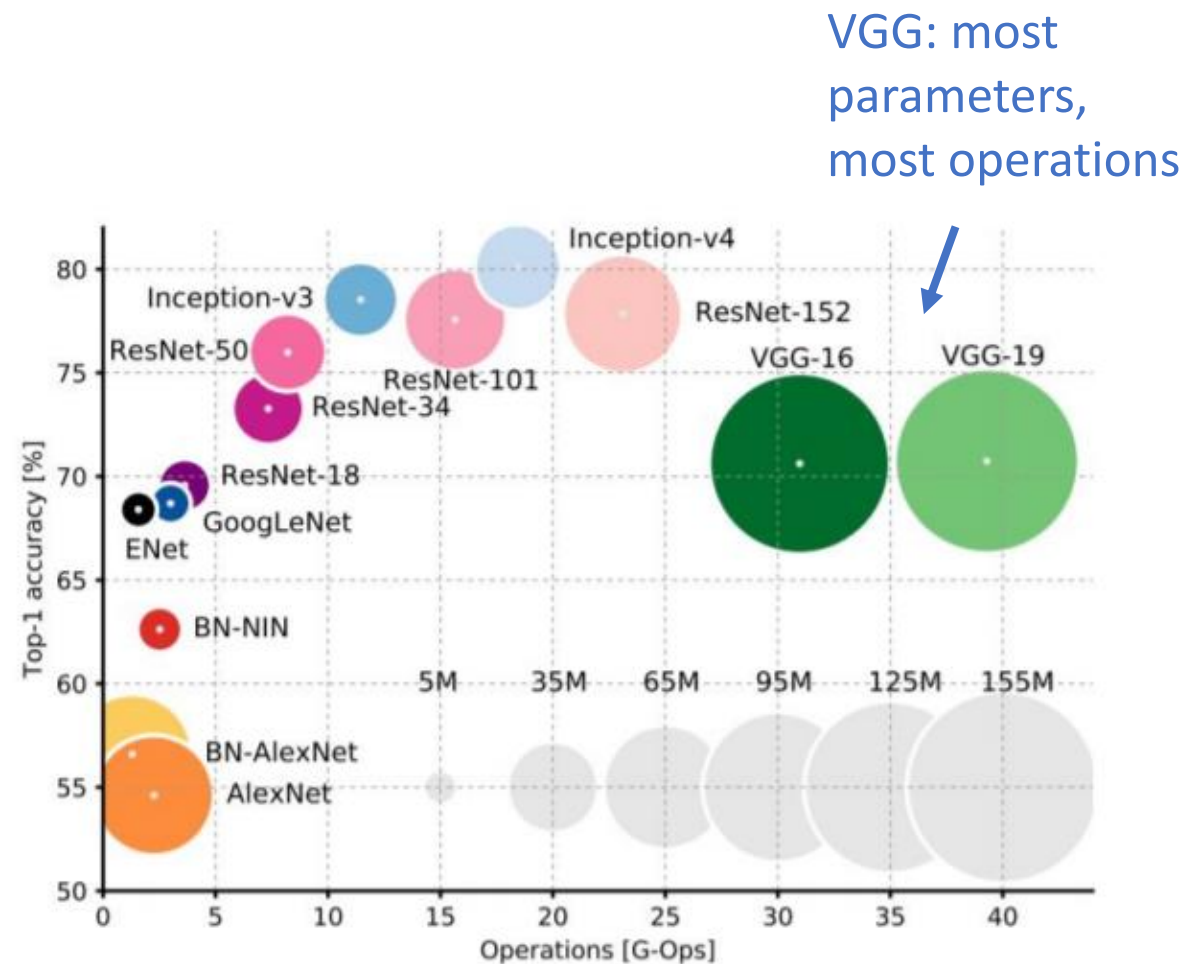
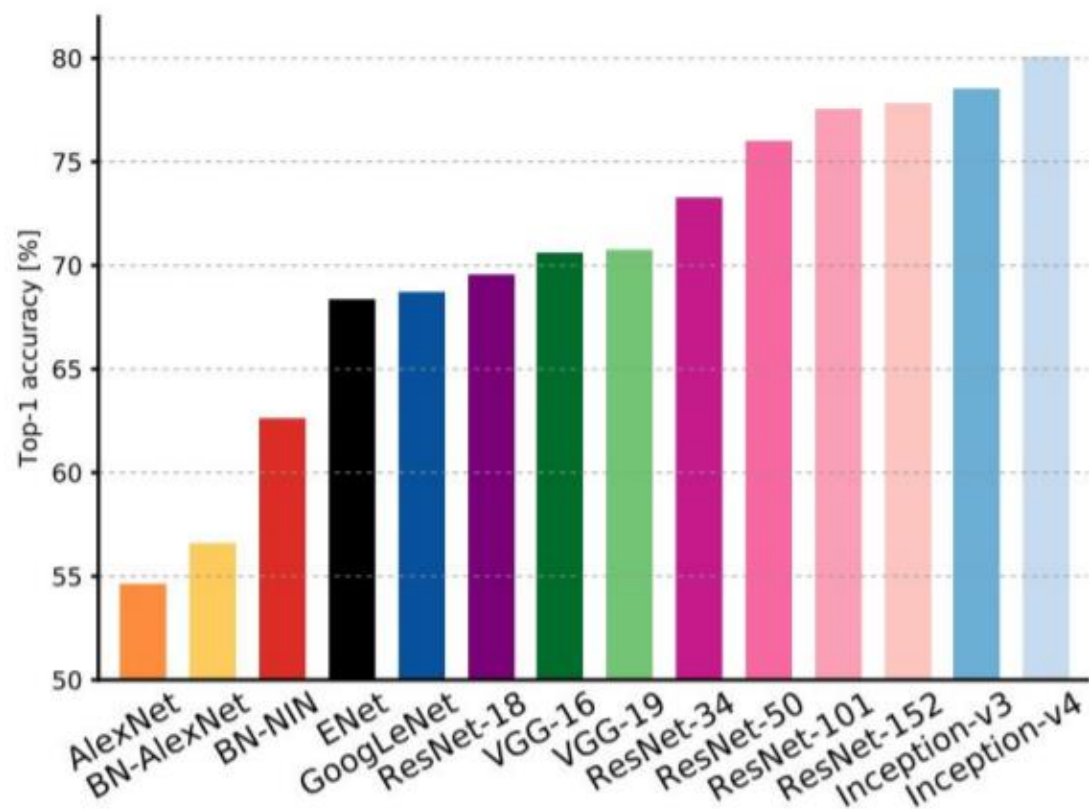
Comparing complexity

Inception-v4: Resnet + Inception!



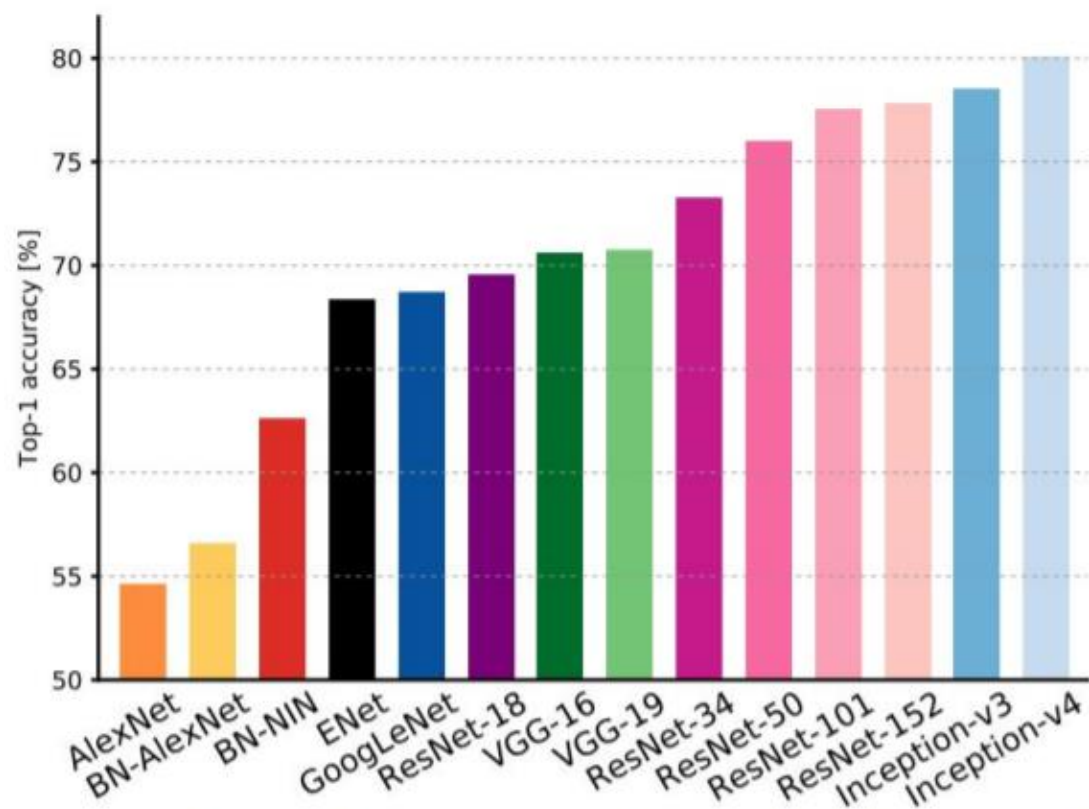
An Analysis of Deep Neural Network Models for Practical Applications, 2017.

Comparing complexity

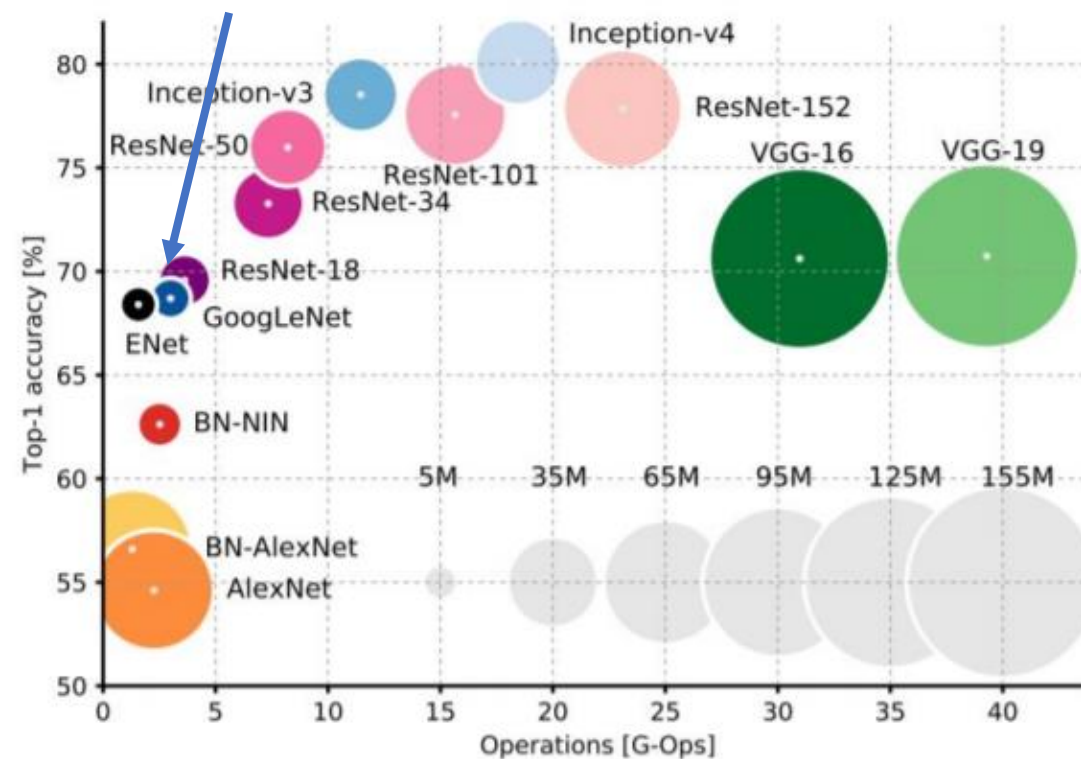


An Analysis of Deep Neural Network Models for Practical Applications, 2017.

Comparing complexity



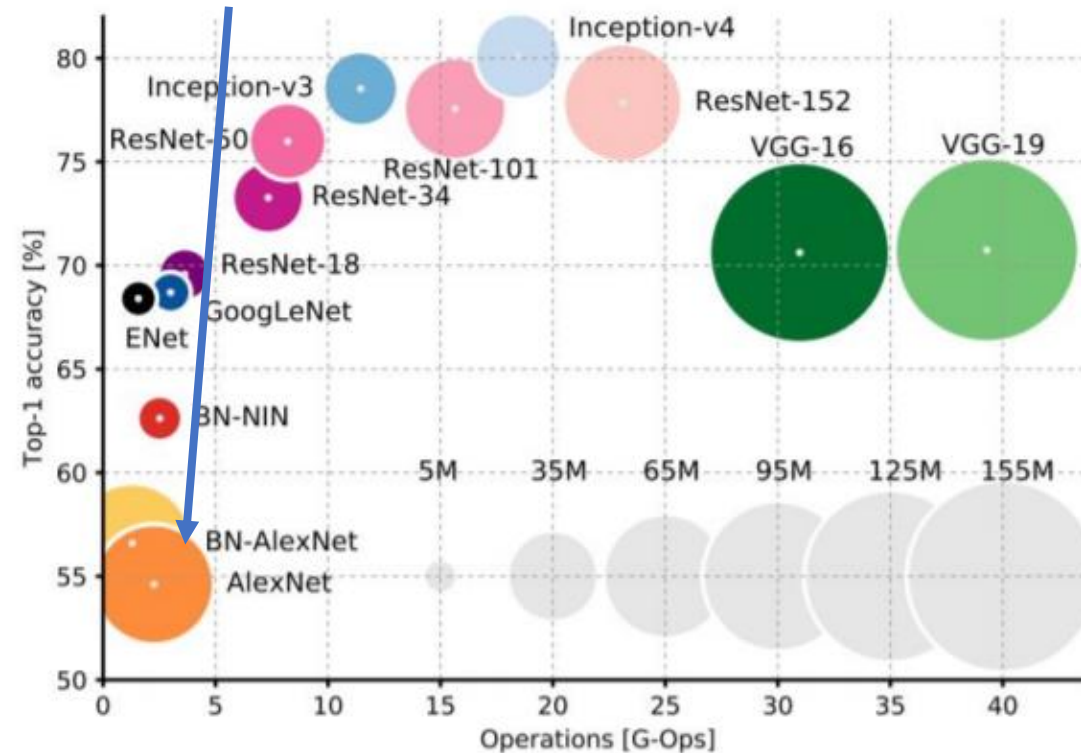
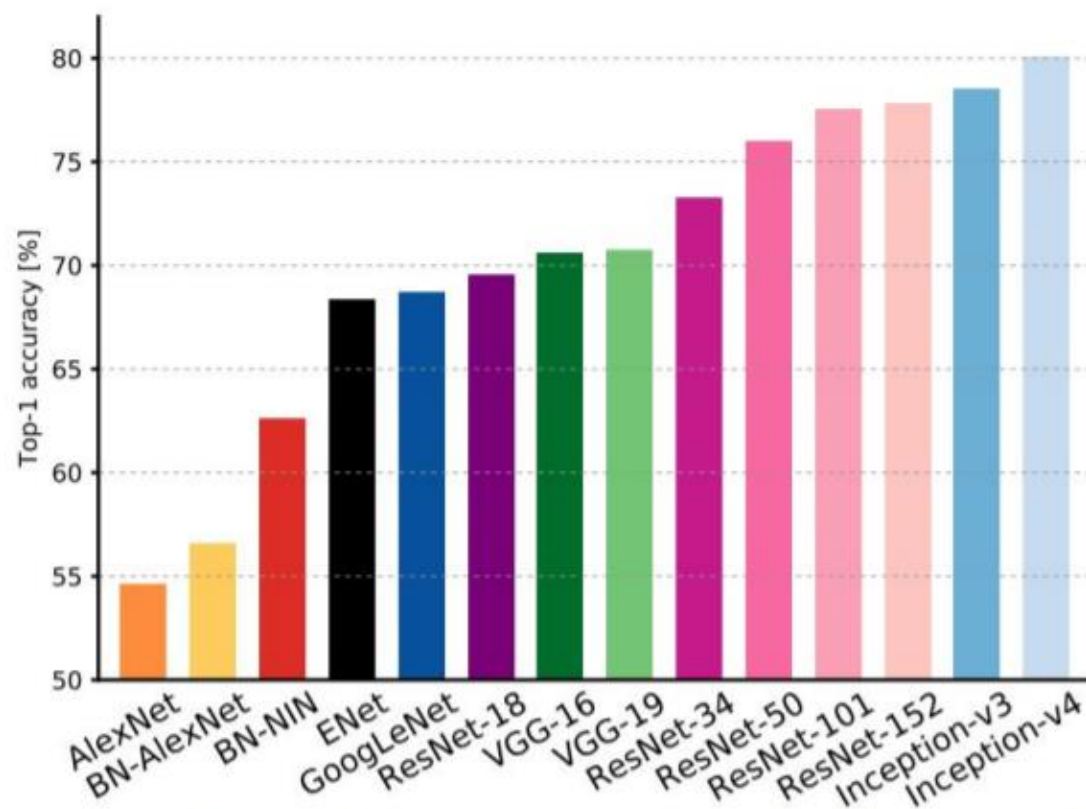
GoogLeNet:
most efficient



An Analysis of Deep Neural Network Models for Practical Applications, 2017.

Comparing complexity

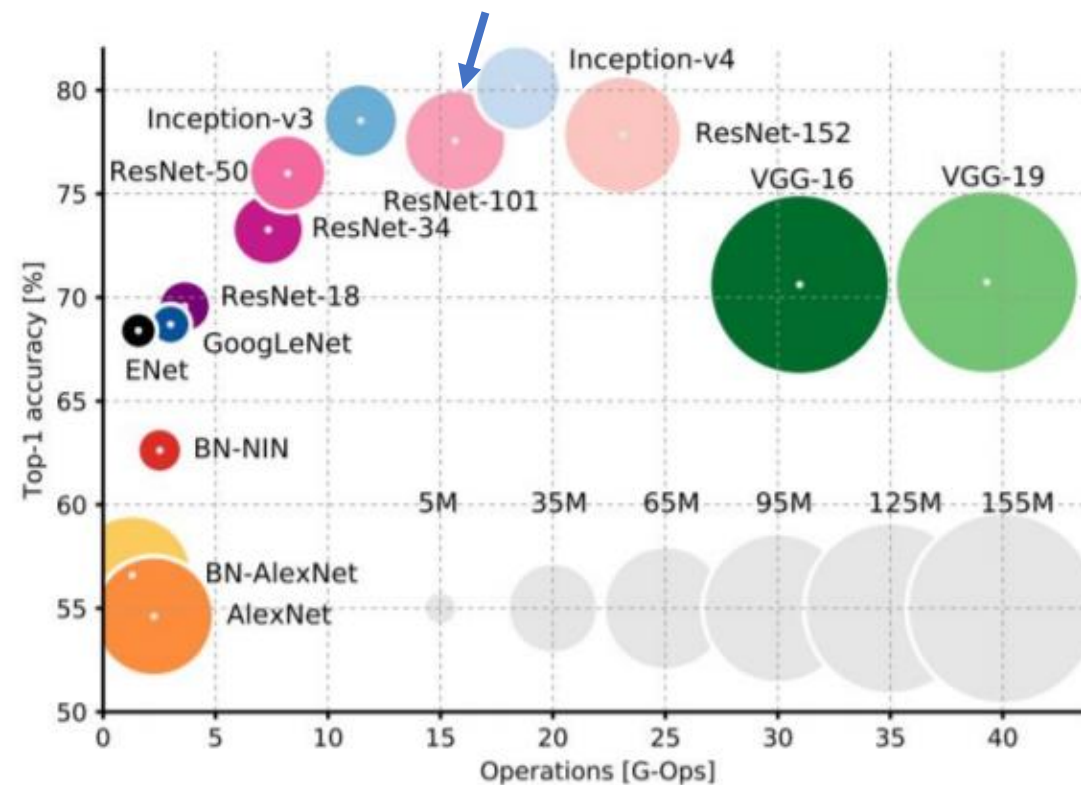
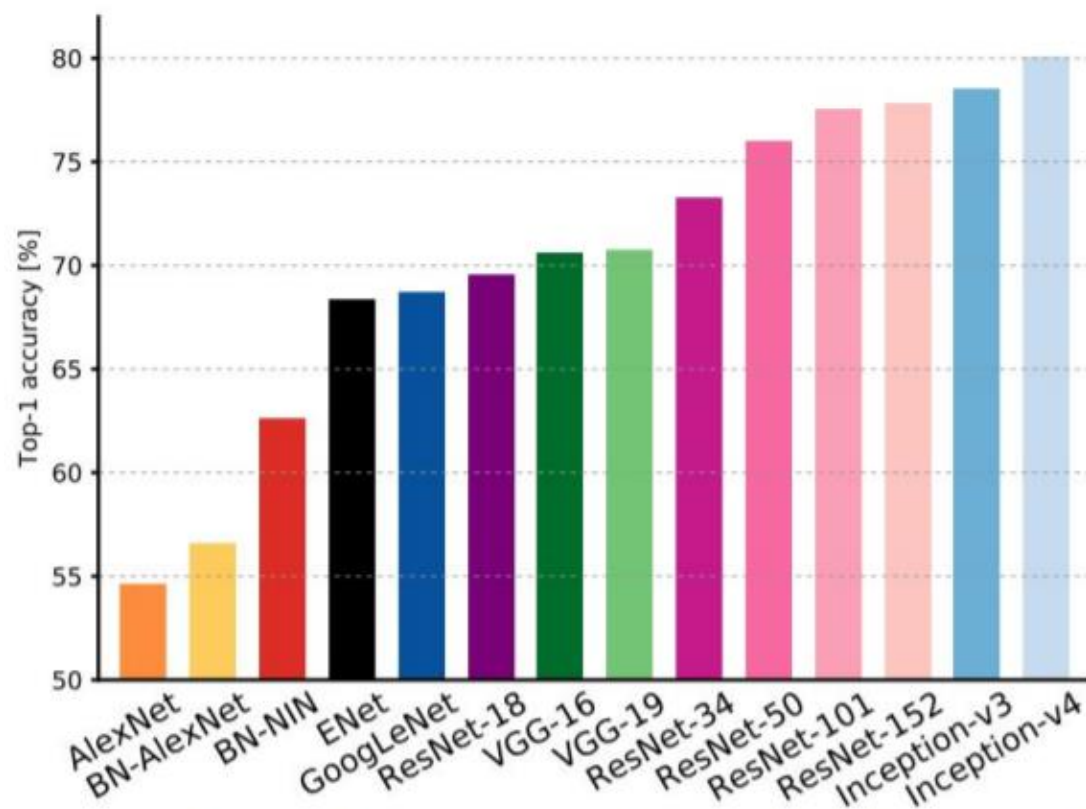
AlexNet:
Smaller compute, still memory
heavy, lower accuracy



An Analysis of Deep Neural Network Models for Practical Applications, 2017.

Comparing complexity

ResNet:
Moderate efficiency depending on
model, highest accuracy



An Analysis of Deep Neural Network Models for Practical Applications, 2017.

Semantic Segmentation

Computer Vision Tasks

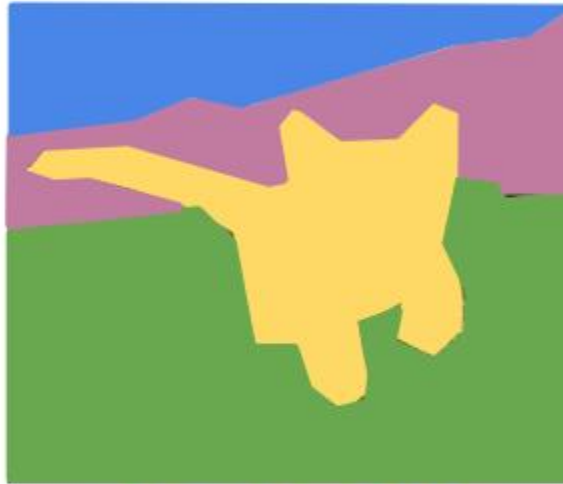
Classification



CAT

No spatial extent

Semantic Segmentation



GRASS, CAT,
TREE, SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Object

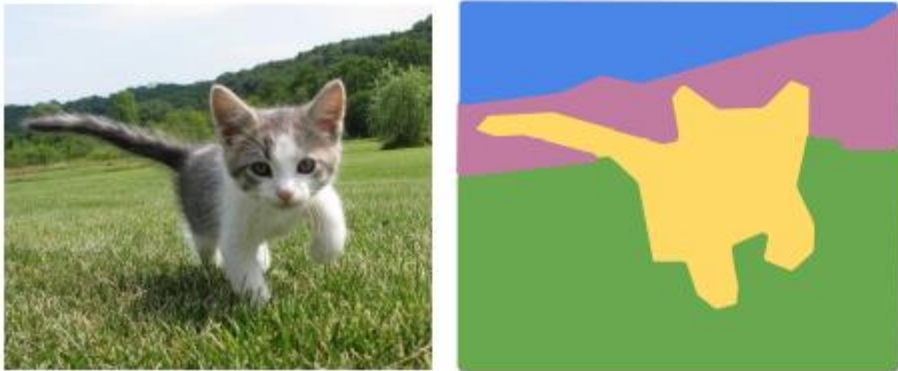
Instance Segmentation



DOG, DOG, CAT

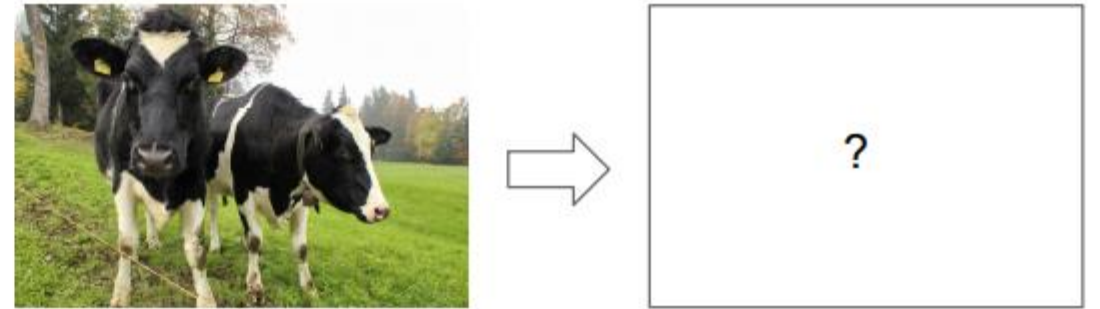
[This image is CC0 public domain](#)

Semantic Segmentation: The Problem



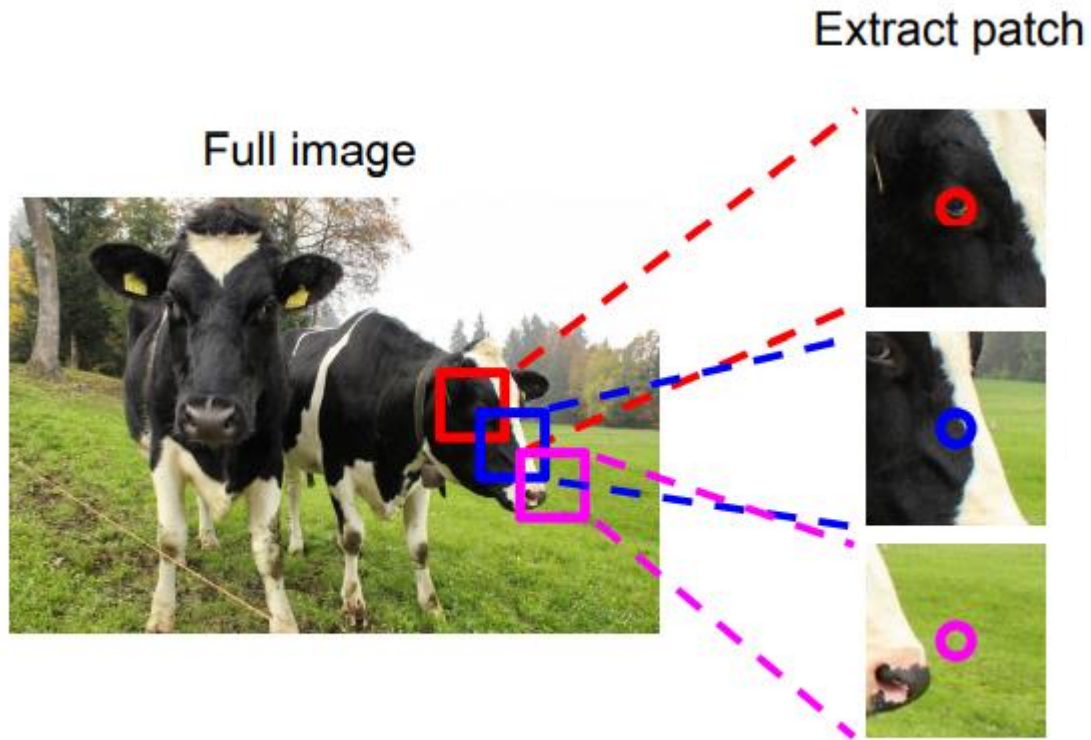
GRASS, CAT,
TREE, SKY, ...

Paired training data: for each training image,
each pixel is labeled with a semantic category.



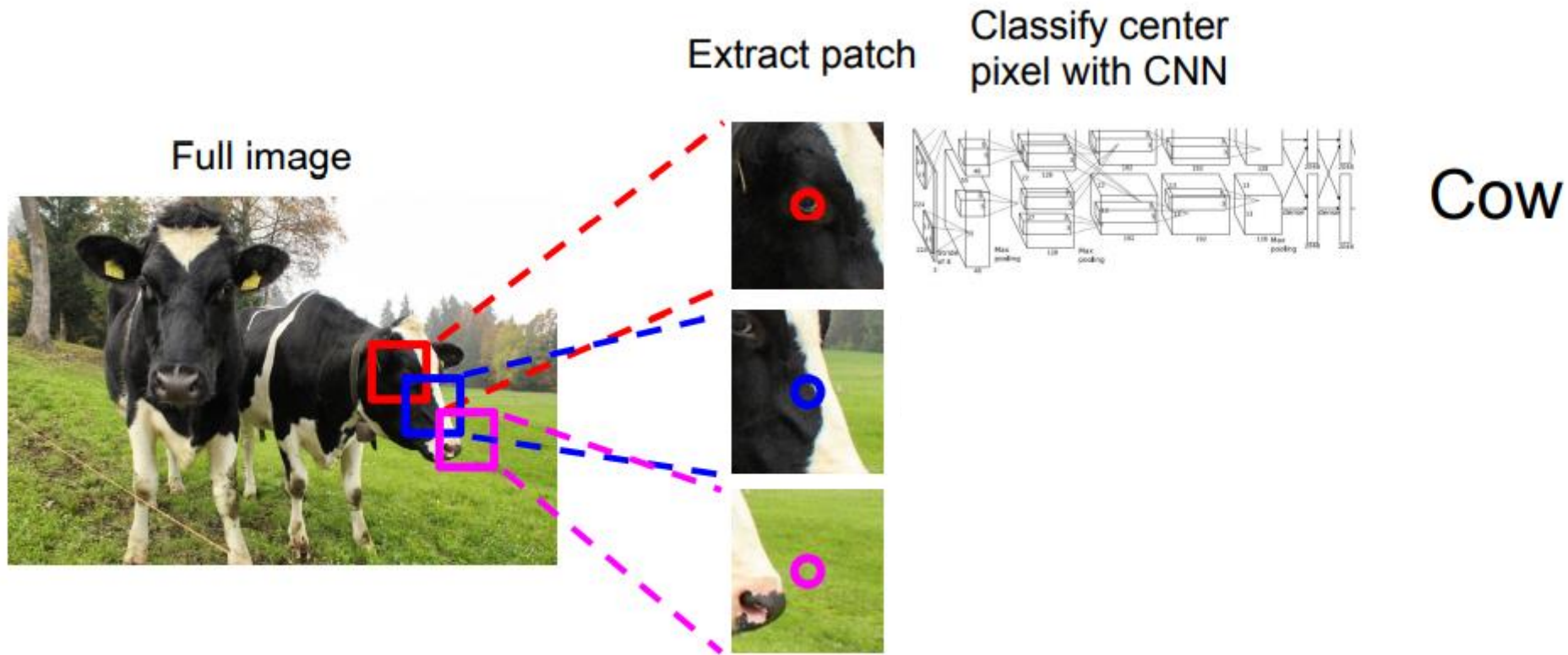
At test time, classify each pixel of a new image.

Semantic Segmentation Idea: Sliding Window

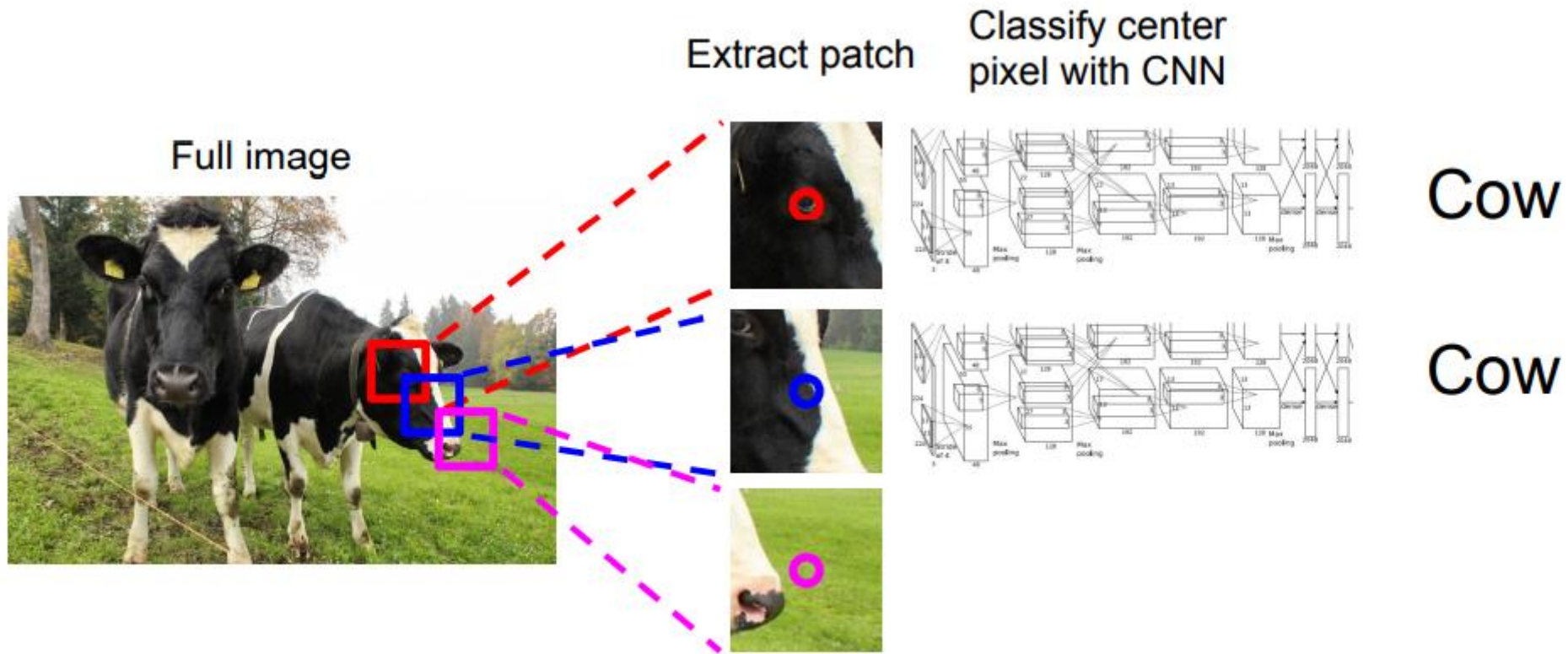


© 2016

Semantic Segmentation Idea: Sliding Window

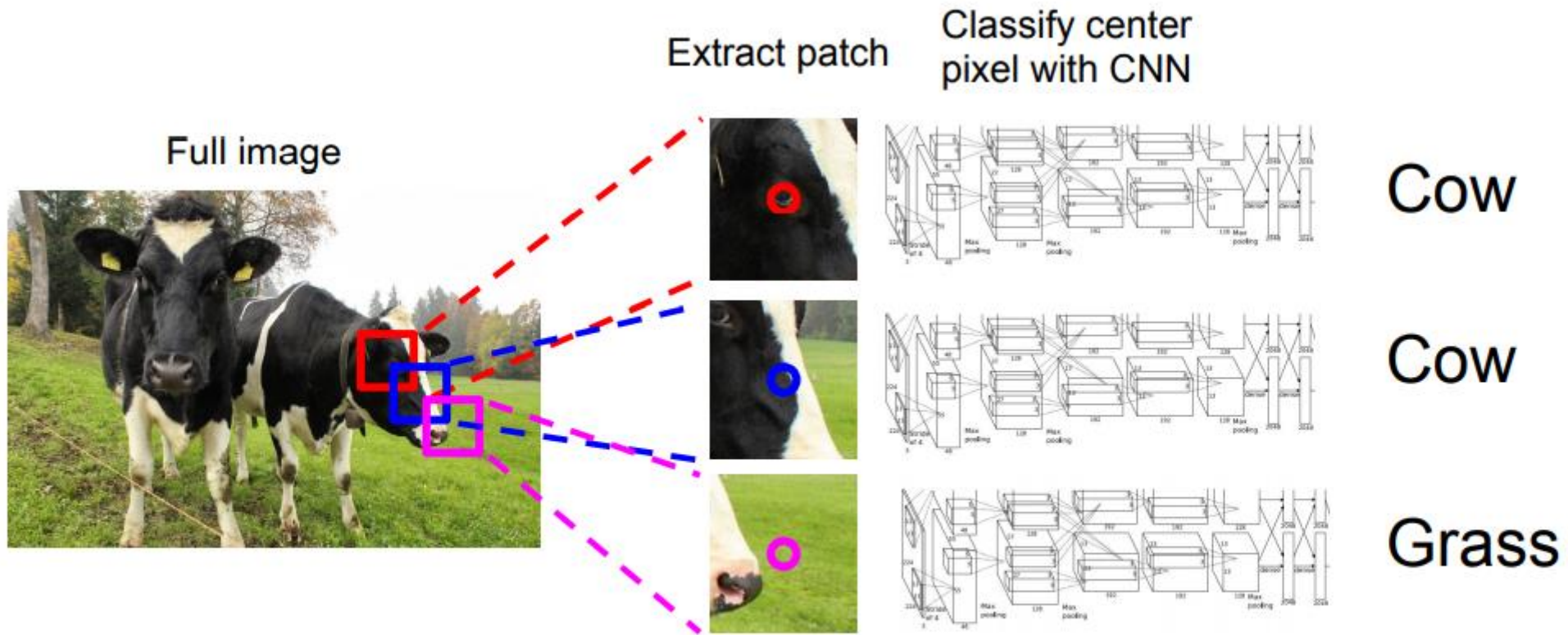


Semantic Segmentation Idea: Sliding Window

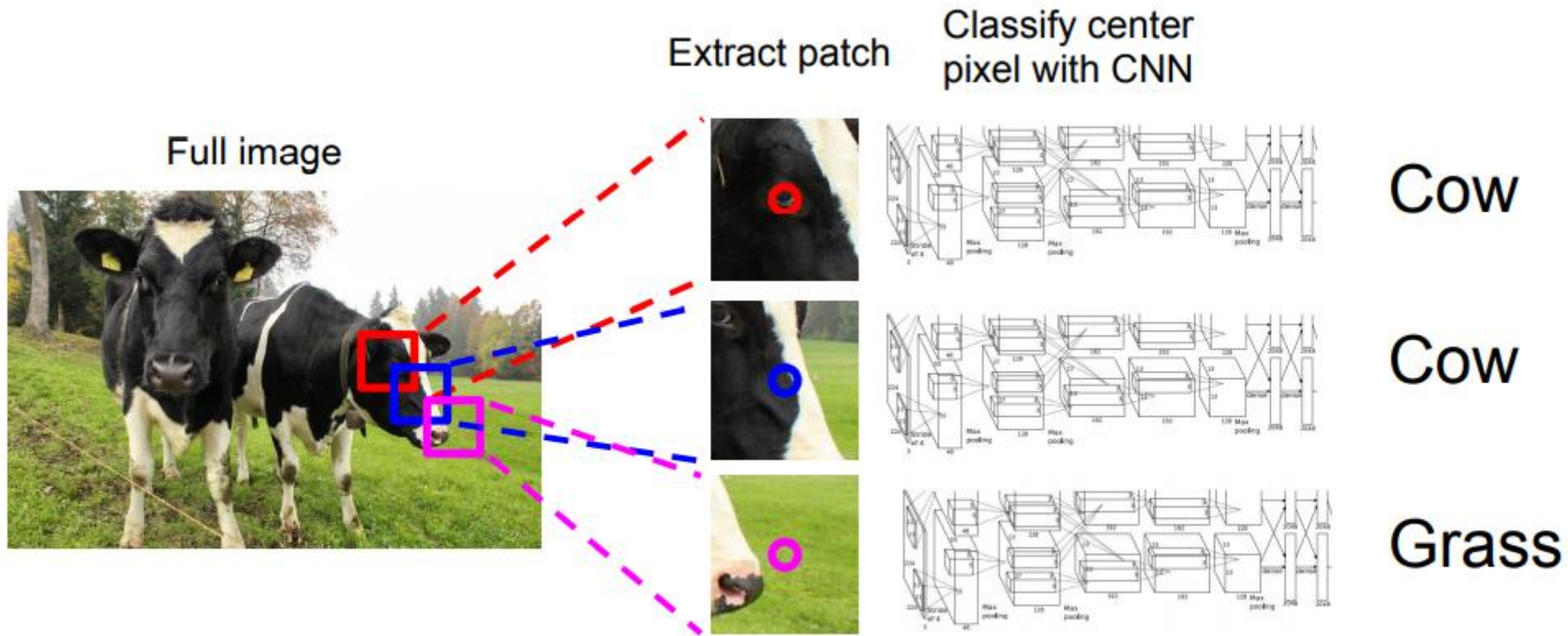


CS231n

Semantic Segmentation Idea: Sliding Window



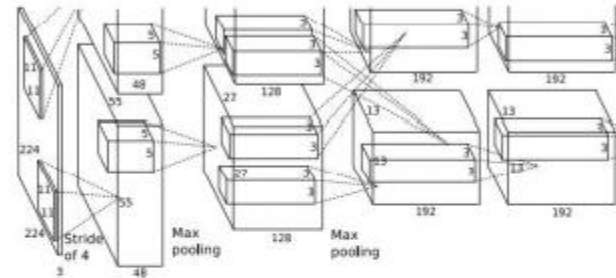
Semantic Segmentation Idea: Sliding Window



Problem: Very inefficient!
Not reusing shared features
between overlapping patches

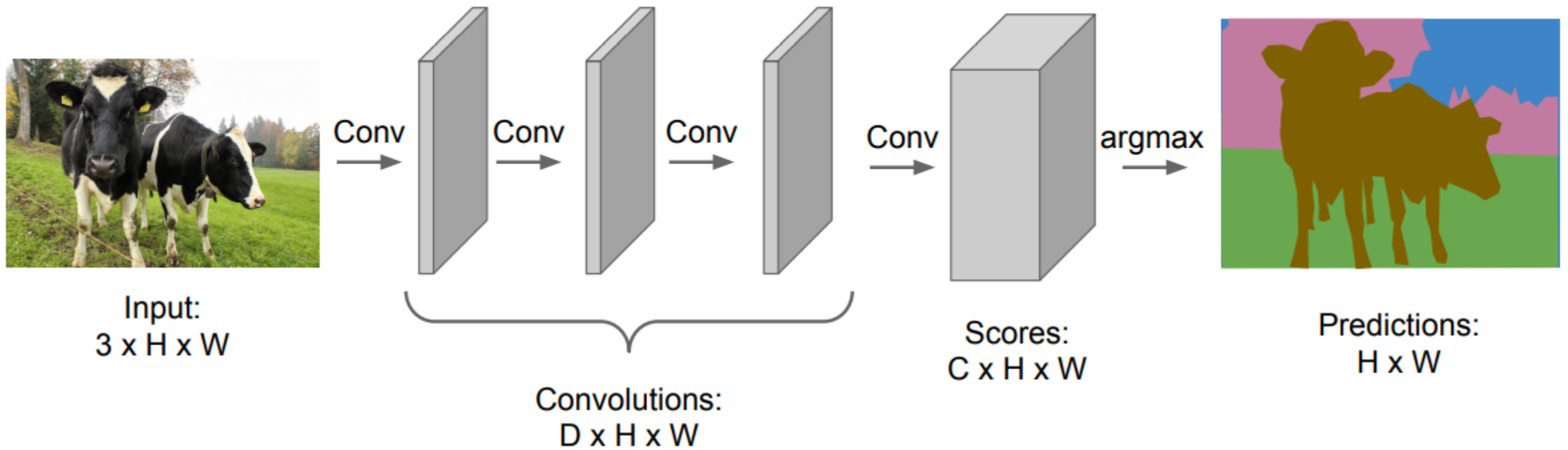
Semantic Segmentation Idea: Convolution

Full image



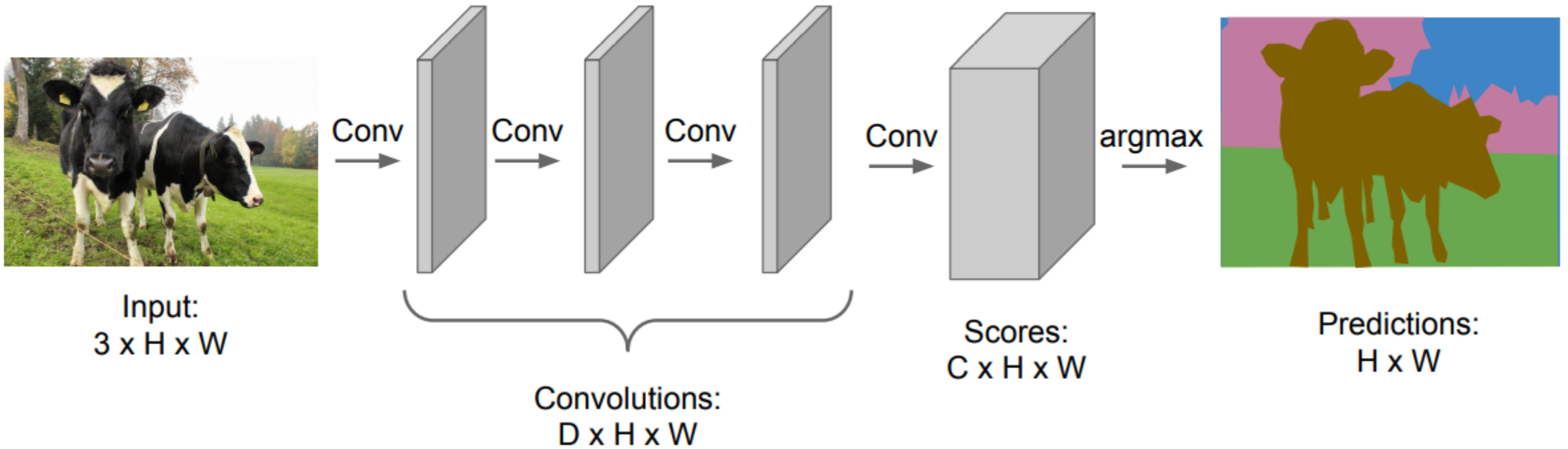
An intuitive idea: encode the entire image with conv net, and do semantic segmentation on top.

Semantic Segmentation Idea: Fully Convolutional



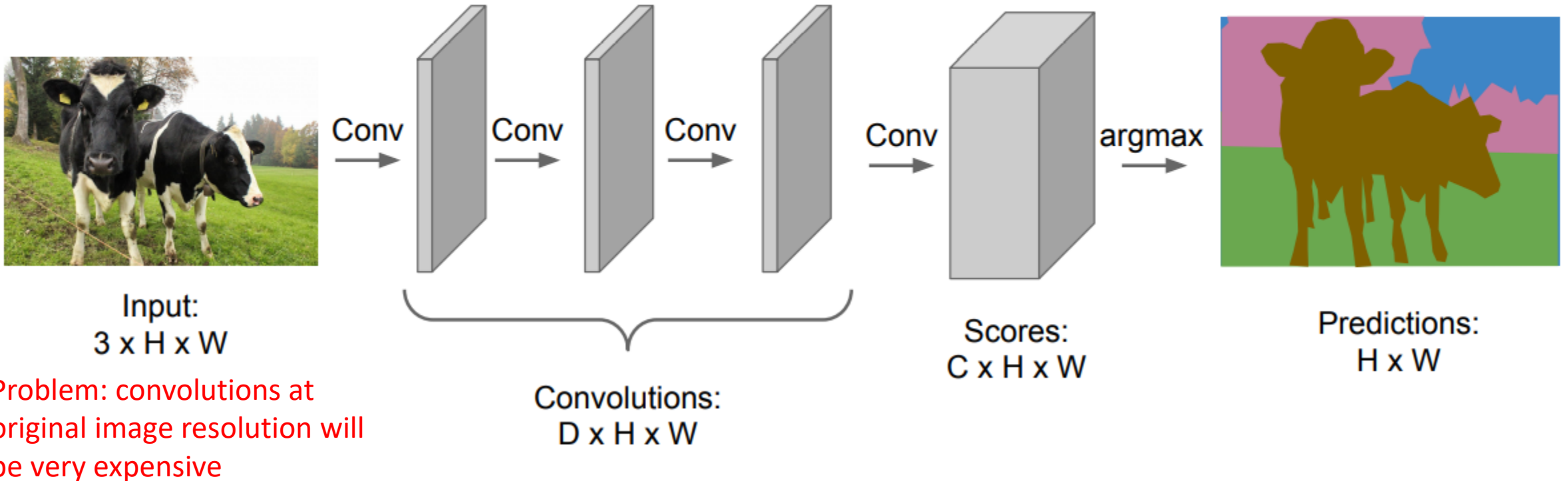
Semantic Segmentation Idea: Fully Convolutional

Design a network with only convolutional layers without downsampling operators to make predictions for pixels all at once!

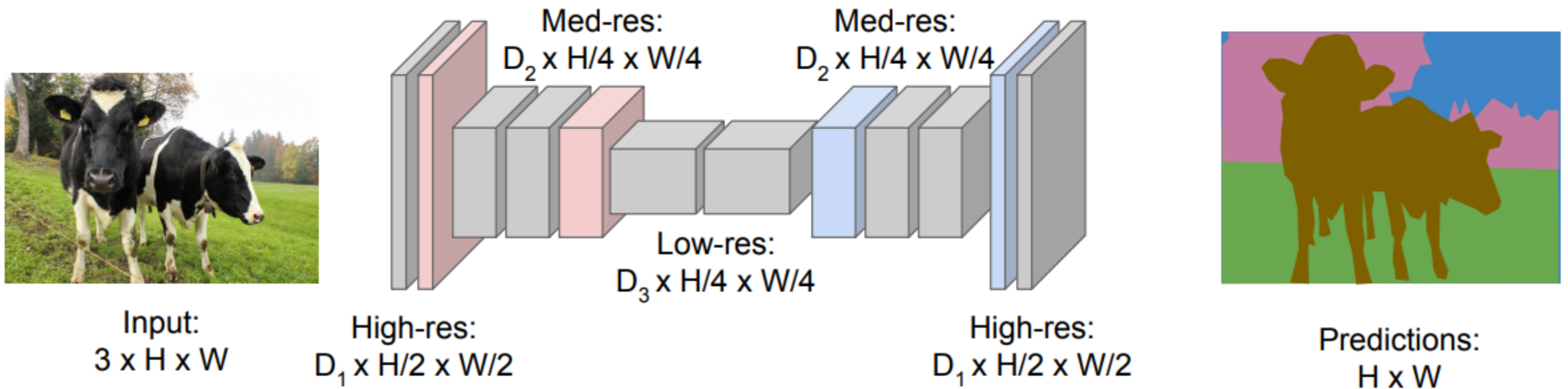


Semantic Segmentation Idea: Fully Convolutional

Design a network with only convolutional layers without downsampling operators to make predictions for pixels all at once!

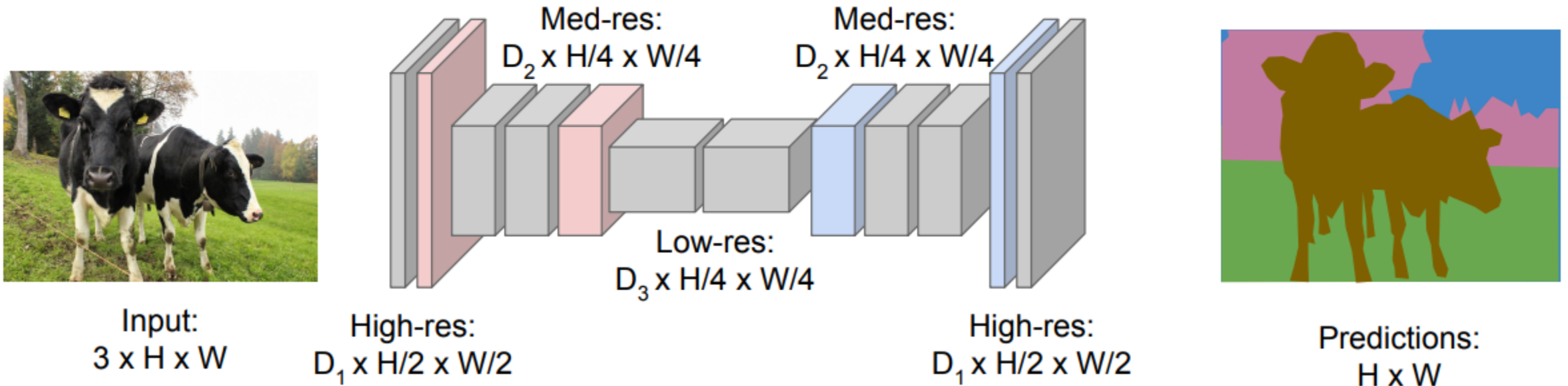


Semantic Segmentation Idea: Fully Convolutional



Semantic Segmentation Idea: Fully Convolutional

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



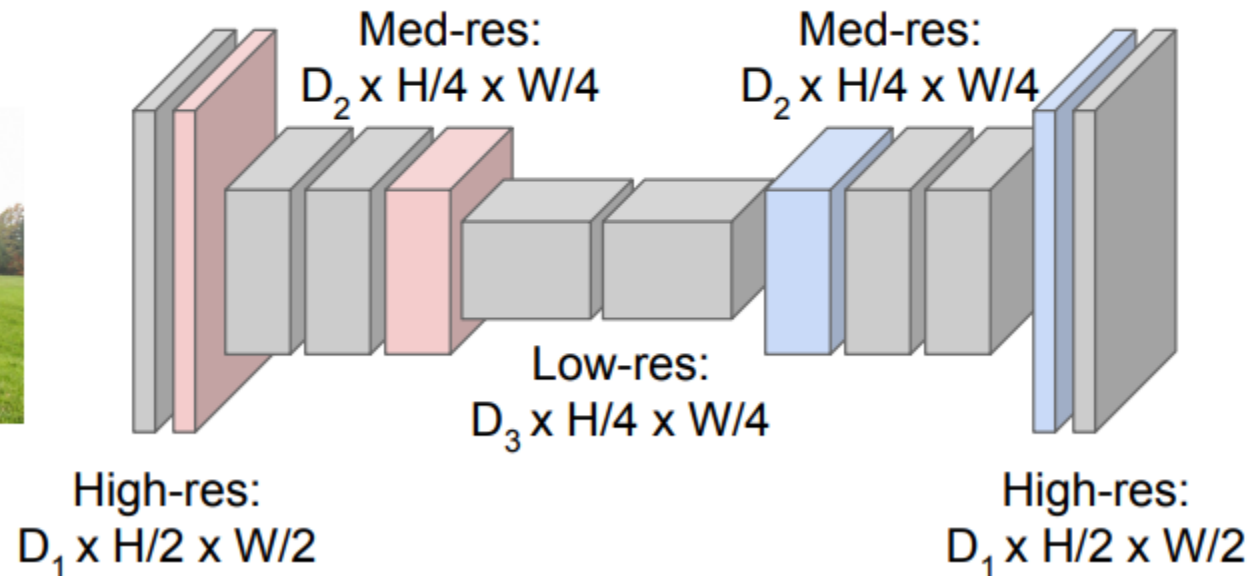
Semantic Segmentation Idea: Fully Convolutional

Downsampling:
Pooling, strided
convolution



Input:
 $3 \times H \times W$

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



Upsampling:
???



Predictions:
 $H \times W$

In-Network upsampling: “Unpooling”

Nearest Neighbor

1	2
3	4



1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Input: 2 x 2

Output: 4 x 4

“Bed of Nails”

1	2
3	4

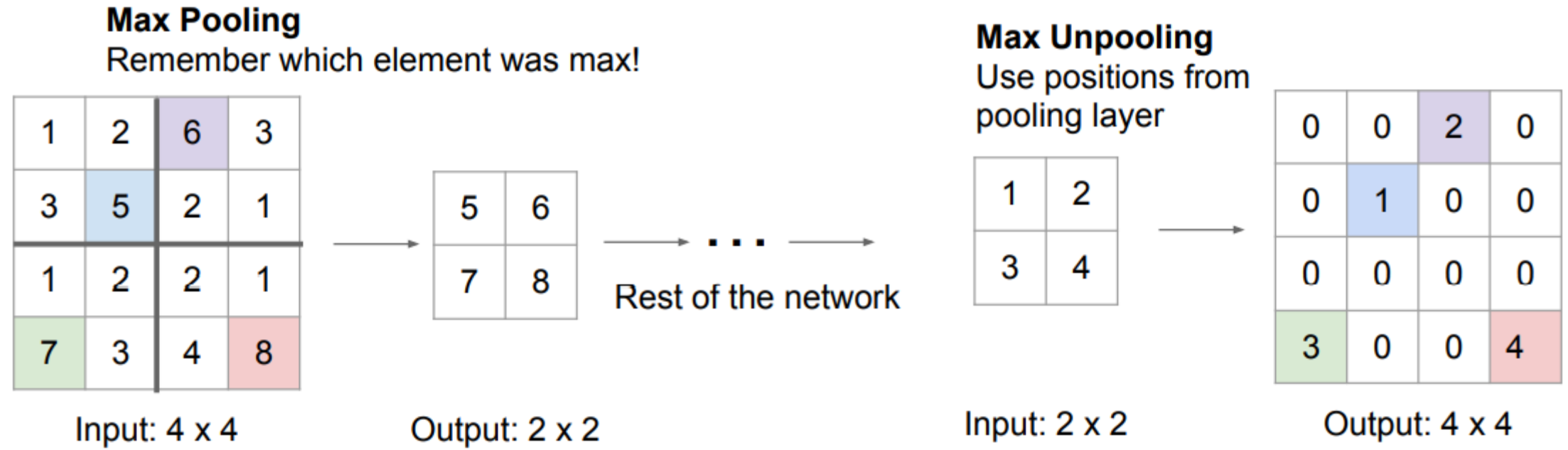


1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

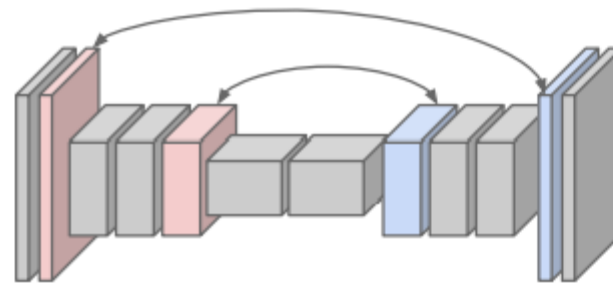
Input: 2 x 2

Output: 4 x 4

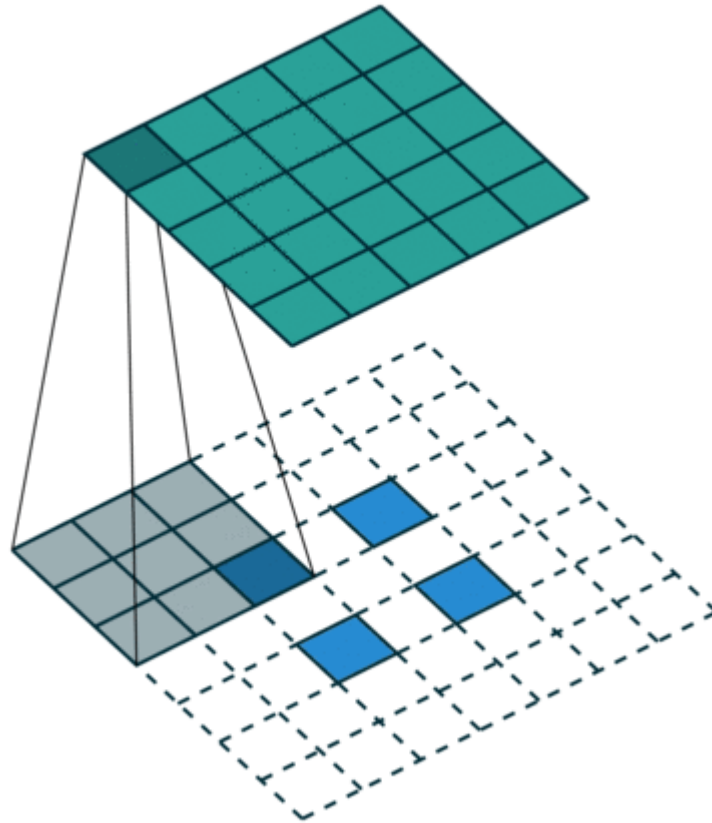
In-Network upsampling: “Max Unpooling”



Corresponding pairs of
downsampling and
upsampling layers



Transposed convolution



If one does a cascade of unpooling + conv operations, we get to the encoder-decoder architecture
Even more refined: Autoencoders with skip connections (aka U-Net)

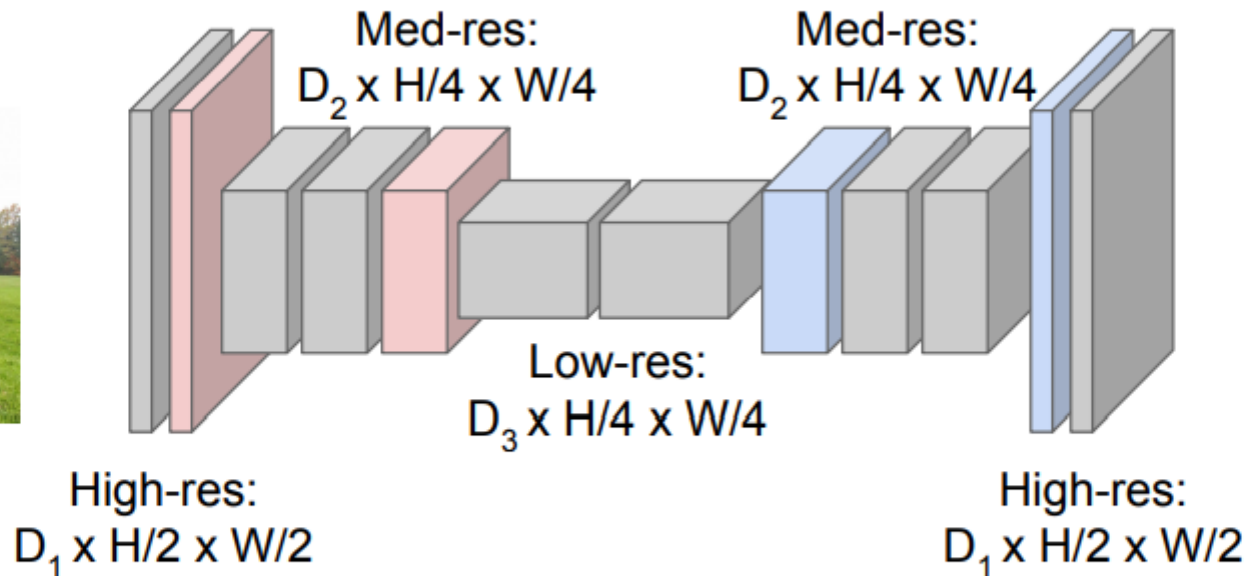
Semantic Segmentation Idea: Fully Convolutional

Downsampling:
Pooling, strided
convolution



Input:
 $3 \times H \times W$

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



Upsampling:
Unpooling or strided
transpose convolution

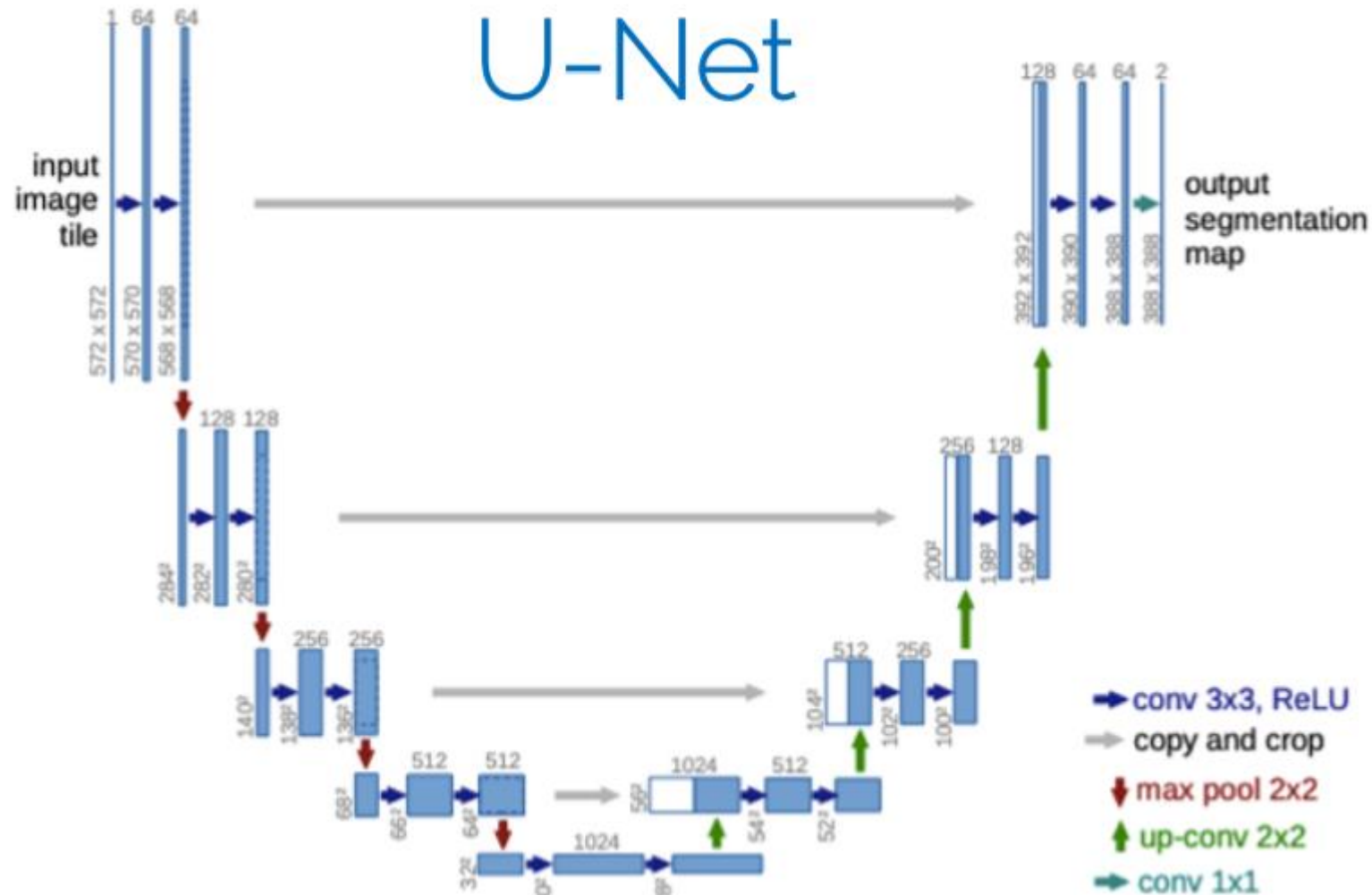


Predictions:
 $H \times W$

Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015

Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

U-Net



U-Net architecture:

Each blue box is a multichannel feature map.

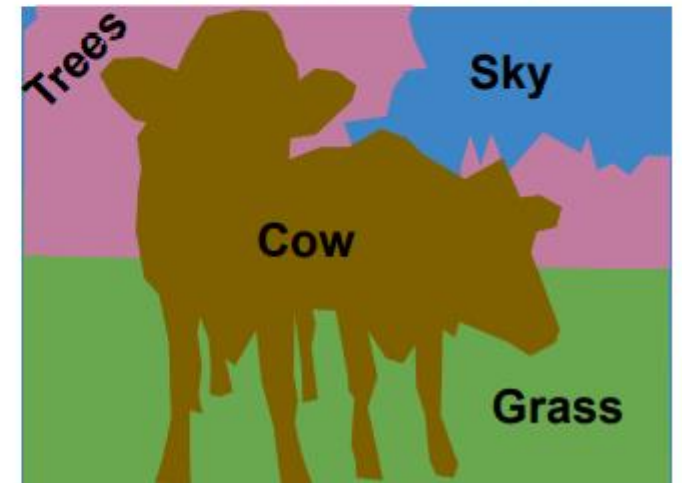
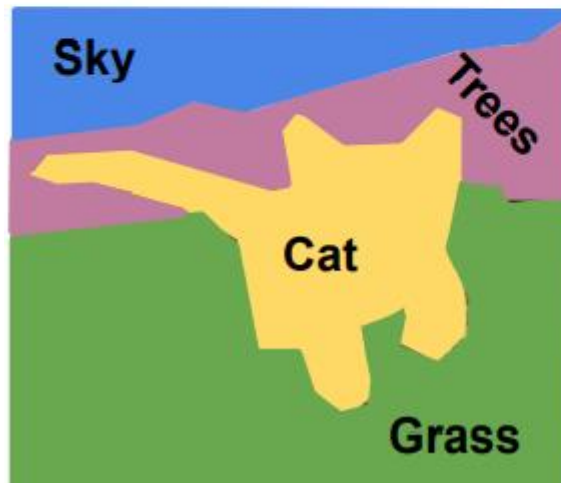
Number of channels denoted at the top of the box .

Dimensions at the top of the box.

White boxes are the copied feature maps.

Semantic Segmentation

Label each pixel in the image with a category label



Semantic Segmentation

Label each pixel in the image with a category label

Don't differentiate instances, only care about pixels

